Docker 从入门到精通

条目	说明
基准版本	Docker Engine 29.x（撰写时最新稳定版为 29.9.2，2026-10-20 发布）、Docker CLI、Buildx、Docker Compose v2（内置 `docker compose` 命令）
核实日期	2026-10-21。版本号、安装步骤、废弃特性与命令行为均依据 Docker 官方文档与官方发布说明核实
适用平台	Ubuntu、Debian 等 Linux 发行版直接安装 Docker Engine；Windows 10 22H2 与更高版本、Windows 11 23H2 与更高版本、macOS 安装 Docker Desktop
目标读者	具备基本 Linux 命令行使用经验，希望系统掌握容器技术的开发者、测试与运维人员
写作约定	默认在 Linux 终端执行示例命令；已标注「适用版本」的行为存在版本差异；所有删除类操作均标明后果，执行前请先确认操作对象

如何使用本教程

全书按「会用、用好、用精」三个部分递进。第一部分（第 1 章至第 4 章）完成安装，并掌握镜像与容器的日常操作；第二部分（第 5 章至第 9 章）解决应用的打包、编排、网络、存储与排错问题；第三部分（第 10 章至第 13 章）覆盖运行时原理、安全加固与生产部署，并以一个完整项目收束。

正式阅读前的三点说明。

1. 示例统一使用两条主线：第 5 章打包的一个 Python Web 应用，第 13 章容器化并发布的完整项目。示例代码与命令均可直接运行。
2. 术语在首次出现处给出定义，重要术语同时附英文原文；附录 A 汇总常用命令，附录 B 汇总全部术语，正文标注核实的版本差异均附适用版本。
3. `console` 代码块表示需要在终端输入的命令，命令下一块为预期输出的节选；`yaml`、`dockerfile`、`json`、`bash` 代码块分别表示对应格式的文件内容。

第一部分 会用 Docker

第一部分完成后，读者应当能够在三种主流平台上安装 Docker，独立获取镜像、运行容器、让外部访问容器中的服务，并完成日常清理。

第 1 章 为什么要容器化

一个应用在开发人员的工作机上运行正常，部署到测试服务器后却报出依赖缺失或版本不符的错误。这一类问题几乎每个团队都遇到过，本章从它的成因出发，说明容器技术如何解决，并建立全书后续章节依赖的概念基础。

1.1 一个反复出现的部署难题

一个典型的 Web 应用不只包含自己的业务代码，还依赖一系列外部条件：特定的解释器或运行时版本、系统共享库、目录权限、环境变量与配置文件。传统部署方式下，这些条件被分散地安装在多台机器上，由手工或脚本维护，由此产生三类常见问题。

问题	表现	成因
环境漂移	同一应用在不同服务器上的行为不一致	各机器的库版本、补丁与配置各不相同
依赖冲突	升级一个组件后，另一个应用无法启动	多个应用共享同一套系统环境
环境重建困难	服务器更换后无法快速恢复服务	环境只有结果、没有可重复的构建过程

这些问题的共同根源在于：应用的交付物只有业务代码本身，而应用真正运行所需的环境没有被一并打包。容器技术的核心思路，就是先解释「环境」这个词，后解释这个解决思路。

1.2 把应用连同环境一起交付

容器（container）是一种由操作系统内核提供的进程级隔离技术：一组进程共享同一个内核，但各自拥有独立的文件系统视图、进程编号空间与网络配置。把一个应用运行所需的全部内容（代码、运行时、依赖库、默认配置）打成一份不可变的软件包，就得到镜像（image）；从镜像启动起来的那个被隔离的进程组，就是容器。

换一个更精确的说法：容器运行的是「应用 + 环境的完整快照」。开发环境与生产环境运行的不再是两份靠猜测对齐的安装结果，而是同一个镜像。

由此带来的收益可以逐条对应到 1.1 节的三类问题。

收益	对应问题	说明
环境一致性	环境漂移	开发、测试、生产使用同一镜像，差异只可能来自配置，不可能来自环境
依赖随应用打包	依赖冲突	每个应用连同自己的依赖库放在独立容器内，互不可见
交付物可重复构建	环境重建困难	镜像由文本化的 Dockerfile 定义（第 5 章专题），重建与否决等价于两个文本差异
启动快、密度高	部署效率	容器只是共享内核的进程组，秒级启动，单机可运行数十个

1.3 容器与虚拟机的区别

虚拟机（virtual machine，VM）通过虚拟机监控器（hypervisor，如 Linux 上的 KVM）虚拟出一套完整硬件，其上运行含独立内核的完整操作系统。容器则直接使用宿主机内核，只隔离进程的视图。两者的差异决定了它们的适用边界。

对比维度	容器	虚拟机
内核	共享宿主机内核	每台虚拟机一个独立内核
隔离强度	内核提供的进程级隔离	硬件级虚拟化隔离
启动时间	通常在毫秒到秒级	通常在数十秒级
磁盘占用	以 MB 计，且多层镜像间共享基础层	以 GB 计
典型用途	微服务、命令行工具、CI 环境等应用承载	强隔离需求场景、异构操作系统需求
在 macOS / Windows 上的形态	通过轻量 Linux 虚拟机运行 Linux 容器	直接运行

需要理解的结论是：容器的隔离强度弱于虚拟机。安全相关的边界约束在第二、三部分有专门讨论（第 11 章）；在此之前，读者可以先记住一个实用判断——容器适合承载应用，而不适合作为不受信代码的强隔离沙盒。

1.4 Docker 与它的三个核心对象

Docker 是一个围绕上述容器技术构建的完整平台，负责把镜像的构建（build）、分发（share）与运行（run）做成一条连续的工作流。全书命令都围绕三个核心对象展开，在此先给出正式定义。

对象	英文	定义	类比
镜像	image	只读的应用打包物，包含文件系统层与启动配置；镜像本身不会运行	程序的可执行文件
容器	container	从镜像启动的运行实例；拥有可写层，随删除而消失	被启动的进程及其内存状态
仓库	registry	集中存放与分发镜像的网络服务；Docker Hub 是 Docker 官方运营的公共仓库	软件包服务器

镜像与容器的关系，最准确的类比是「模板与实例」：一份镜像可以同时启动多个内容相同、各自带独立可写层的容器；容器被删除后，它运行期间产生的数据（未挂载出的部分）随之消失，而镜像不受影响。

1.5 Docker 的组成与架构

一次 `docker run` 命令背后，实际参与工作的组件最少有四个。

组件	角色	说明
Docker CLI	命令行客户端	即终端中的 `docker` 命令，负责解析参数并把请求发送给守护进程
Docker daemon	服务端守护进程	即 `dockerd` 进程，负责容器、镜像、网络与存储的管理，Docker Engine 指的就是它
containerd	容器运行时管理组件	负责镜像内容的存储与容器生命周期，自 Engine v29 成为新安装环境的默认镜像存储（详见第 10 章）
容器运行时	创建进程	以 runc 为代表，实际调用内核接口创建被隔离的进程

执行链可以概括为：CLI 把命令通过本机套接字发给 daemon，daemon 经 containerd 调用运行时完成实际的容器创建。相关网络请求（如拉取镜像）由 daemon 直接完成。

本节只需要建立一条印象：`docker` 命令的绝大多数操作，是「向本机 daemon 发请求」。因此第 9 章与第 11 章关于 daemon 配置与安全的内容，作用范围都覆盖全部容器。调用链的完整机制见第 10 章。

1.6 本章小结

- 传统部署难题的根源是环境未被随应用打包；容器通过「镜像 + 内核级隔离」对此提供了系统性的解决。
- 容器与虚拟机的本质差异是进程级隔离与硬件级隔离，容器更轻，虚拟机更隔离。
- Docker 平台的操作围绕镜像、容器、仓库三个对象展开；命令行请求最终都由本机 daemon 处理。
- 下一章完成安装并运行第一个容器，验证环境。

第 2 章 安装并打通第一个容器

安装是全书一切操作的前提。本章覆盖 Linux、Windows 与 macOS 三种平台，并解释安装后的关键配置项；完成本章后，读者能够运行 `hello-world` 容器并确认整个调用链正常工作。

2.1 版本选择与发行渠道

Docker Engine 有两个更新渠道：稳定渠道（stable）发布正式可用版本，测试渠道（test）发布候选版本，测试渠道包含实验性特性且可能存在不兼容变更。学习与生产均应使用稳定渠道。

撰写时的版本状况直接给出结论。

对象	状况
Docker Engine 稳定线	29.x 系列，最新稳定版 29.9.2（2026-10-20）
Docker Desktop	图形化的桌面产品，内置 Engine、Compose、Buildx，面向 Windows 与 macOS
Docker Compose	CLI 层面受支持的是 Compose v2 与 2025 年发布的 Compose v5；v5 功能上与 v2 等价，主要增加了官方 Go SDK。两者都以 Compose Specification 解释 Compose 文件

一个需要预知的版本差异：Engine v29 起最低支持的 API 版本被上调，本机 CLI 与 daemon 版本差异过大时，`docker version` 可能打印「client version is too old」一类错误。随 Docker Desktop 或官方仓库同步升级通常不会遇到该问题；本教程基准 29.x，文中标注的版本行为均以 29.x 为准。

2.2 在 Linux 上安装（Ubuntu 为例）

Linux 上安装 Docker Engine 的推荐方式是配置 Docker 官方软件源（apt 源或 dnf/yum 源），由系统包管理器负责安装与升级。以下按 Ubuntu 步骤展开；Debian 的步骤与之结构相同，仅仓库地址中的发行标识不同（Debian 支持版本为 13 Trixie 与 12 Bookworm）。其他发行版及离线二进制安装方式，见官方安装文档（docs.docker.com/engine/install）。

支持的 Ubuntu 版本为 26.10、26.04 LTS、24.04 LTS、22.04 LTS，64 位架构。若曾安装过发行版自带或来源不明的 Docker 相关包（例如 `docker.io`、`docker-compose`），需先移除，否则会与官方包冲突：

```
$ sudo apt remove docker.io docker-compose docker-compose-v2 docker-doc podman-docker containerd runc
```

此命令对未安装的包会提示「未安装」，不影响后续步骤。接下来两步为配置官方 apt 源。第一步添加官方 GPG 签名密钥（GPG，GNU Privacy Guard，一种数字签名体系，apt 用它校验软件源内容未被篡改）：

```
# 添加 Docker 官方 GPG 密钥
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

第二步把仓库地址写入 apt 源列表：

```
# 添加 Docker 官方 apt 源
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

然后安装。默认安装最新稳定版：

```
$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

各包的作用如下表。安装完成即同时具备 Engine、构建器与 Compose，无需再单独安装其他组件。

包名	作用
docker-ce	Docker Engine 本体（daemon 端）
docker-ce-cli	Docker CLI 客户端
containerd.io	containerd 及其依赖（runc 等）的打包集合
docker-buildx-plugin	Buildx 构建器（提供 `docker buildx` 及现代的 `docker build` 实现，见第 5 至第 6 章）
docker-compose-plugin	Compose v2，以 `docker compose` 子命令形式提供

安装完成后，服务会被注册并启动。以下命令用于查看与启动（systemd 为 Linux 通用的服务管理器）：

```
$ sudo systemctl status docker
$ sudo systemctl start docker
```

最后运行官方测试镜像完成验证：

```
$ sudo docker run hello-world
```

该命令的执行过程是：客户端向本机 daemon 请求运行 `hello-world` 镜像；本机不存在该镜像，daemon 从 Docker Hub 拉取；镜像载入后运行其中唯一的容器，容器打印确认信息后退出。屏幕上出现以 Hello from Docker 开头的确认段落，即为本命令成功的标志。看到该输出，说明 CLI、daemon、镜像存储与容器运行时四个环节全部工作正常。

2.3 非 root 用户使用 Docker

Linux 上，daemon 的管理接口 `/var/run/docker.sock` 默认只允许 root 与 docker 组成员访问。把用户加入 docker 组即可免 sudo 使用 `docker` 命令：

```
$ sudo usermod -aG docker $USER
```

退出当前终端并重新登录后生效。可用 `id -nG` 检查自己是否已在 docker 组中。

必须明确说明安全后果：docker 组成员拥有的权限实质上等同 root，因为任何 docker 组成员都可以向 daemon 发起任意请求，包括把宿主机任意目录挂载进容器。生产服务器上的组成员决策，请结合第 11 章的安全内容综合判断。个人开发机上的常见做法是加入 docker 组。

2.4 在 Windows 上安装 Docker Desktop

Windows 不直接安装 Linux 版 Engine，而是安装 Docker Desktop。它在后台维护一个轻量 Linux 虚拟机，容器在该虚拟机内运行；容器操作与 Linux 一致。

以推荐的 WSL 2 后端（Windows Subsystem for Linux 第 2 代，微软提供的轻量虚拟化层）为例，系统要求如下。

要求	具体值
Windows 版本	Windows 10 64 位：Enterprise、Pro 或 Education，版本 22H2（build 19045）及以上；Windows 11 64 位：Enterprise、Pro 或 Education，版本 23H2（build 22631）及以上
内存	8 GB 系统内存
WSL	WSL 2.1.5 或更高（使用增强容器隔离则需 2.6 或更高）
BIOS/UEFI	需启用硬件虚拟化
安装模式	默认按用户安装（不需要管理员权限）

需要注意两个边界：Windows 家庭版只能运行 Linux 容器；运行 Windows 原生容器需要 Pro 或 Enterprise 版并使用管理员级别的安装模式。安装包从 docker.com 下载，按向导完成安装后启动 Docker Desktop 即可；在终端中可直接使用 `docker` 命令。详情以官方页面（docs.docker.com/desktop/setup/install/windows-install）为准。

2.5 在 macOS 上安装 Docker Desktop

与 Windows 相同，macOS 上安装 Docker Desktop，容器在产品内部管理的 Linux 虚拟机中运行。要求为 macOS 最近的三个大版本之一、至少 4 GB 内存；Apple silicon 与 Intel 芯片分别选择对应安装包。建议同时安装 Rosetta 2 转译层，用于兼容少量仍在使用 amd64 架构的命令行工具：

```
$ softwareupdate --install-rosetta
```

由于容器一律运行在虚拟机内的 Linux 内核上，Linux 容器镜像在三种平台上的行为一致；这正是「一次打包、随处运行」如何在桌面平台实现的机理。平台差异主要出现在文件系统挂载性能与端口安全性细节（分别见第 8 章与第 11 章对应小节）。

2.6 安装后的基础配置

以下配置全部可选，逐步给出三个最常用的点。

第一，服务随系统自启。发行版包安装的 Engine 默认开机自启，可用以下命令确认或调整状态：

```
$ sudo systemctl is-enabled docker
$ sudo systemctl enable docker
```

第二，日志限制。容器日志默认不轮转，长期运行会占满磁盘。多数环境建议在 daemon 配置文件 `/etc/docker/daemon.json` 中设置默认日志轮转策略（该机制详见第 9 章）：

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

含义为每个容器最多保留 3 个日志文件、每个不超过 10 MB。修改后重启服务使配置生效：`sudo systemctl restart docker`。注意该配置只对新启动的容器生效，已存在的容器重建后才应用新默认值。

第三，镜像加速。中国大陆网络访问 Docker Hub 较慢时，可为 daemon 配置 registry mirrors，地址列表依赖所选服务商，配置键为 `daemon.json` 中的 `"registry-mirrors"`，具体地址以服务商提供的为准，本教程不给出具体地址以免误导。校园网与内网环境的常见做法是自建镜像仓库，推送与拉取方法见 5.6 节。

2.7 升级与卸载

Linux 上升级与安装是同一条命令：先 `sudo apt update` 后重新执行 2.2 节的安装命令，apt 会自动升级到新版。卸载则先移除包再删除数据：

```
$ sudo apt remove docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
$ sudo rm -rf /var/lib/docker /etc/docker
```

第二条命令删除镜像、容器、卷等全部本地数据，且该删除不可恢复。镜像、容器、卷存放在 `/var/lib/docker/` 中，卸载软件包不会自动清空它们；是否执行第二条命令，取决于之后是升级还是彻底移除。

2.8 常见安装问题速查

现象	常见原因	处理
permission denied while trying to connect to the Docker daemon socket	当前用户既非 root 也不在 docker 组	用 sudo，或按 2.3 节加入 docker 组后重新登录
Cannot connect to the Docker daemon	daemon 未启动	`sudo systemctl start docker` 后查看 `sudo systemctl status docker`
Docker Desktop 启动失败并提示虚拟化	BIOS/UEFI 中未启用硬件虚拟化	进固件设置开启；Windows 另需启用 WSL 2 功能
apt 安装报签名校验错误	GPG 密钥未正确导入	核对 2.2 节第一步的路径与权限（keyrings 目录、`.asc` 文件）

2.9 本章小结

- Engine 有 stable 与 test 两个渠道，学习与生产使用 stable；撰写时稳定线为 29.x。
- Linux 推荐用官方 apt/dnf 源安装五个 docker 相关包，一次装齐 Engine、CLI、containerd、Buildx 与 Compose。
- Windows 与 macOS 通过 Docker Desktop 使用等价的容器能力；Linux 容器在虚拟机内运行，三种平台行为一致。
- 加入 docker 组可免 sudo，但该权限等同 root，需按环境性质决策。
- 完成 `docker run hello-world` 即表示调用链各环节正常，可以进入日常操作。

第 3 章 容器与镜像的日常操作清单

Docker 的高频操作可以概括为一组「动词」：对镜像做拉取、查看、删除，对容器做创建、启停、进入、删除，辅以查看与清理工具。本章按操作对象组织全部日常命令，并解释每个参数的确切含义；本章之后遇到其他命令时，对照本表即可快速上手。

3.1 镜像的获取与管理

先建立引用格式概念。镜像的完整名称由仓库名与标签组成：

```
[仓库地址/]命名空间/镜像名[:标签]
```

标签（tag）是同一镜像不同版本的别名，如 `ubuntu:24.04`。省略标签时默认为 `latest`。Docker Hub 官方镜像省略命名空间，如 `nginx`。给镜像生成的内容指纹是摘要（digest，形如 `sha256:...`），内容相同则摘要相同，摘要不随标签改名而变化。生产环境建议始终写明确的标签；内容固定的依赖镜像可以锁定摘要（用法见第 6 章与第 12 章）。

常用镜像命令如下。

命令	作用	说明
`docker pull ubuntu:24.04`	拉取镜像	从仓库下载到本地
`docker image ls`	列出本地镜像	别名 `docker images`；加 `-a` 显示全部
`docker image inspect ubuntu:24.04`	查看镜像详细配置	输出 JSON，含每层信息与启动配置
`docker image rm ubuntu:24.04`	删除本地镜像	别名 `docker rmi`
`docker search alpine`	在 Docker Hub 搜索镜像	输出名称、描述与星数
`docker image save -o ubuntu.tgz ubuntu:24.04`	导出镜像为归档文件	配合 `load` 可用于无外网环境迁移
`docker image load -i ubuntu.tgz`	从归档导入镜像	与 `save` 成对使用

删除时的一个细节：如果目标镜像被某个容器（即使已停止）引用，删除会报错，先删除引用它的容器，或者给 `docker image rm` 加 `-f` 强制删除（此时镜像的标签被移除，内容仍保留直至无引用）。

3.2 运行容器：docker run 详解

`docker run` 是使用频率最高的命令，一次性完成「从镜像创建容器并启动进程」两步，等价于 `docker create` 加 `docker start`（两者将在 3.3 节说明）。完整形式为：

```
docker run [选项] 镜像[:标签][命令][参数...]
```

选项作用于 daemon 创建容器的方式，末尾的「命令与参数」则会替换镜像的默认启动命令。最常用的选项如下表，其中端口、环境变量与数据挂载三项在第 4 章展开。

选项	含义
`--name web`	为容器指定名称；省略时 daemon 自动生成（如 `jovial_curie` 一类的随机名）
`-d`	detached，后台运行并立刻返回容器 ID；省略则容器日志直接打在当前终端
`--rm`	容器退出后自动删除，适合一次性任务
`-i` 、 `-t`	`-i` 连接标准输入，`-t` 分配伪终端；连写 `-it` 才能使用交互式 shell
`-e KEY=value`	注入环境变量，可重复使用
`-p 8080:80`	端口映射，发布容器服务到宿主机
`-P`	大写 P，把镜像 `EXPOSE` 声明的端口全部映射到宿主机随机端口
`-v 数据卷或路径:容器内路径`	挂载数据卷或宿主机目录
`-w /app`	指定容器内的工作目录
`--hostname demo`	设置容器主机名（容器内 `hostname` 命令所见）
`--restart 策略`	设置重启策略（见 4.4 节）
`--network 网络名`	指定容器加入的网络（见 8.2、8.3 节）
`--user 1000`	指定容器内进程运行的用户（见 11.2 节）
`--memory 512m` 、 `--cpus 1.5`	限制内存与 CPU 配额（见 3.3、12.2 节）
`--entrypoint /bin/xxx`	覆盖镜像的入口点（见 5.3 节）
`--pull always`	强制使用仓库中的最新镜像，忽略本地副本

两个代表性示例。第一个运行后台 Web 服务（nginx 为常用 Web 服务器镜像，其宿主服务目录为 `/usr/share/nginx/html`）：

```
$ docker run -d --name web -p 8080:80 nginx
4f2e1a0b9c8d...
```

输出的一长串字符是容器 ID 前 12 位。第二个运行一次性交互式环境：

```
$ docker run -it --rm ubuntu:24.04 bash
root@7a1f2b3c4d5e:/#
```

进入容器内 shell 后可随意观察，`exit` 退出即容器停止并因 `--rm` 被删除。`-it` 与 `--rm` 的组合是临时实验的惯用法。

这里值得停下来解释一个初学者高频困惑：为什么有的镜像直接 `docker run` 就一直运行（如 nginx），有的却立刻退出。容器随主进程退出而退出。nginx 镜像的主进程是常驻的 nginx 服务，所以容器保活；ubuntu 镜像的默认命令是打印一行帮助信息后就退出的短命令，所以容器秒退。让 ubuntu 这类镜像保持心跳，需要显式指定常驻命令，例如 `docker run -d ubuntu:24.04 sleep infinity`（4.4 节还有更好的方案）。

3.3 容器生命周期管理

容器有明确的生命周期状态：汇总如下。

状态	触发	说明
created	`docker create`	已创建未启动；`docker run` 内部先进入此状态
running	`docker start` 、 `docker run`	主进程正常运行
paused	`docker pause`	进程被内核冻结（SIGSTOP），恢复用 `docker unpause`
restarted	daemon 或策略重启	此状态短暂
exited	主进程退出	保留已停止的容器，数据仍在可写层
dead	异常	daemon 回收失败时进入，无法正常操作

日常命令如下表。「优雅停止」指先向主进程发送 SIGTERM 信号（进程可正常收尾的超时宽限期，默认 10 秒），超时后升级为 SIGKILL（内核立即结束进程，不可阻塞）。

命令	作用
`docker start web`	启动已创建或已停止的容器
`docker stop web`	优雅停止；超时参数用 `--timeout`（`--time` 已废弃，废弃声明见 Engine v28.0）
`docker restart web`	先 stop 再 start
`docker kill web`	直接发送 SIGKILL，不做宽限等待
`docker pause web` / `docker unpause web`	冻结 / 恢复容器内全部进程
`docker rm web`	删除已停止的容器；删除运行中的加 `-f`
`docker rm $(docker ps -aq)`	删除所有容器（含运行中，需 `-f`；删除不可恢复，先 `docker ps -a` 核对清单）

读懂容器的退出码（exit code）是排错的第一步：运行 `docker ps -a` 或 `docker inspect web` 都能看到。常见值与含义如下。

退出码	含义
0	正常退出
1	应用自身报错退出
125	daemon 层出错（常见于 `docker run` 参数错误）
126	启动命令存在但不可执行（权限）
127	启动命令不存在
137	进程收到 SIGKILL，常见于超过内存上限被内核杀掉（OOM，Out Of Memory，内存耗尽）
143	进程收到 SIGTERM 后退出，即一次正常的 stop

3.4 进入容器执行命令

需要在运行中的容器内观察或修改状态时，用 `docker exec`：

```
$ docker exec -it web bash
```

`exec` 在容器内新开一个进程执行给定命令，`-it` 保证可交互。基础镜像未必提供 bash，Debian 系镜像用 `bash`，Alpine 系镜像用 `sh`。在容器内退出 shell 不会终止容器，因为主进程仍在。`exec` 的另一个典型用途是一次性执行单条命令并取回输出：

```
$ docker exec web env
```

另一个相关命令是 `docker attach web`，它把当前终端「接回」容器的主进程，常用于观察立即退出的容器到底打印了什么。注意 attach 期间终端与主进程直接相连，若主进程本身是交互式 shell，按 Ctrl-C 会把它终止。这两个命令的差异值得用一张表确认。

对比	docker exec	docker attach
新建进程	是，独立于主进程	否，直接连主进程
影响容器主进程	否	可能（取决于主进程对输入处理的反应）
典型用途	临时排查、单条命令	观察主进程直接输出

3.5 查看容器状态与信息

命令	作用
`docker ps`	列出运行中容器；`-a` 包含已停止的；`-q` 只取容器 ID
`docker ps --filter name=web`	按名称过滤；也可 `--filter status=exited`
`docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"`	自定义列输出
`docker inspect web`	查看容器全部配置与状态（JSON）
`docker logs web`	查看日志；`-f` 跟踪输出，`--tail 100` 限行数，`-t` 加时间戳
`docker stats`	实时资源占用（CPU、内存、网络、块设备 IO）；`--no-stream` 输出一次即退出
`docker top web`	查看容器内进程列表
`docker events`	实时事件流；`--filter type=container` 过滤
`docker diff web`	查看容器可写层相对镜像的文件增删改

`inspect` 的输出很长，常用 `--format` 模板取关键字段（该模板为 Go template 语法）：

```
$ docker inspect -f '{{.State.Status}} {{.State.ExitCode}}' web
running 0
```

3.6 在容器与宿主机之间复制文件

`docker cp` 完成双向复制，语法与系统 cp 一致：

```
$ docker cp web:/etc/nginx/nginx.conf ./nginx.conf
$ docker cp ./config.yaml web:/etc/app/config.yaml
```

第一行从容器复制到宿主机，第二行反向。第二个示例执行前要求目标路径存在，路径不存在时 cp 会失败。它是手工调试与临时取文件的利器；正式的配置交付手段仍是镜像与挂载（第 4、8 章）。

3.7 将容器变更保存为镜像

在容器内手工做出的修改（安装软件、改配置）可以固化为新镜像：

```
$ docker commit -m "add curl" -a "ops-team" web myimage:debug
```

`--message` 与 `--author` 提供说明信息。commit 是有明确适用边界的工具：适合留存调研过程的现场快照，不适合作为正式的构建产物。理由是它记录的层没有任何构建指令信息，无法重放、难以审查、不能增量更新；正式产物应当由 Dockerfile 构建（第 5 章），使「怎么得到这个镜像」成为可审查的文本。

3.8 清理不再使用的对象

Docker 的多种对象（容器、镜像、卷、网络、构建缓存）都会持续占用磁盘。清理命令分两类：按对象清理与全量清理。

命令	清理对象	后果说明
`docker container prune`	全部已停止的容器	已停止容器中的未挂载数据一并消失
`docker image prune`	无标签引用的悬空镜像	影响小，常用
`docker image prune -a`	全部未被容器引用的镜像	未备份的本地镜像会需重新拉取
`docker system df`	查看磁盘占用明细	只读，不带清理动作
`docker system prune`	停止容器 + 悬空镜像 + 无用网络 + 构建缓存	组合清理
`docker system prune -a --volumes`	再加全部未被引用的镜像与未被挂载的卷	最激进；数据库数据可能一并删除，生产环境禁用

清理策略的合理默认是：日常使用 `docker system df` 观察，占用升高时先 `docker image prune` 与 `docker container prune`；`-a` 与 `--volumes` 需在有明确清理目标或开发机环境下使用。

3.9 日常操作速查表

任务	命令
拉取镜像	`docker pull <镜像>:<标签>`
运行容器	`docker run -d --name <名字> -p <宿主端口>:<容器端口> <镜像>`
查看全部容器	`docker ps -a`
进入容器	`docker exec -it <名字> bash`（或 `sh`）
查看日志	`docker logs -f <名字>`
查看资源	`docker stats`
停止 / 重启	`docker stop <名字>` / `docker restart <名字>`
删除容器	`docker rm <名字>`
删除镜像	`docker rmi <镜像>`
文件复制	`docker cp <名字>:<路径> .`
磁盘占用	`docker system df`
组合清理（执行前须确认对象）	`docker system prune`

3.10 本章小结

- 镜像引用由仓库、命名空间、镜像名与标签组成；`latest` 是默认标签而非「最新版」的意思，生产使用应写明确标签。
- `docker run` 是创建并启动的合一操作；容器随主进程退出而退出，选镜像时先想清楚它的主进程是什么。
- 生命周期命令围绕状态机运转；stop 与 kill 的差异是一次可收尾的停止与一次强制停止。
- 排错的第一信号是退出码与环境信息，`docker ps -a`、`docker inspect`、`docker logs` 构成最小诊断组合。
- commit 适合留存手工调试快照；可重复的正式产物需要 Dockerfile，这正是第 5 章的主题。

第 4 章 从外部访问容器

容器默认与外界隔离，其中的服务外部无法访问，需要的数据也无法留存。本章解决三个具体的接入问题：让外部访问容器端口、把配置传进容器、把数据存出容器；并补上容器保活与自动重启两个运行期策略。

4.1 端口映射

先说原理。默认网络（bridge 模式）下，容器拥有一个宿主机不可路由的内网 IP。通过隐藏中间层（安装时已随 Docker 部署的 iptables 规则），`-p` 选项会把宿主机对应端口收到的流量转发到容器端口（网络细节见第 8 章）。因此「发布端口」指的是建立宿主机端口到容器端口的映射。

语法按需要选择。

格式	示例	含义
`-p 宿主端口:容器端口`	`-p 8080:80`	宿主机 8080 转发到容器 80
`-p 宿主IP:宿主端口:容器端口`	`-p 127.0.0.1:8080:80`	只在回环地址接受连接，外部网络无法访问
`-p 容器端口`	`-p 80`	宿主端口随机分配
`-P`	单独使用	镜像 `EXPOSE` 声明的端口全部随机映射
指定协议	`-p 53:53/udp`	默认为 tcp，UDP 需显式写 `/udp`

验证示例。从第 3 章的 nginx 容器继续（若已删除，重新执行 3.2 节的命令）：

```
$ docker run -d --name web -p 8080:80 nginx
$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Server: nginx/1.2x.x
```

「HTTP/1.1 200 OK」即 nginx 的正常响应。在另一台机器上该服务通过 http://宿主机IP:8080 访问。

三条实用规则。第一，宿主端口冲突是运行期最常见的报错（「port is already allocated」一类），可用本机 socket 工具查占用（Linux）：

```
$ ss -ltnp | grep 8080
```

第二，公网服务器上尽量少开端口并优先使用回环绑定；管理后台一类的服务强烈建议 `-p 127.0.0.1:端口:容器端口`，或交由反向代理统一暴露。第三，一个安全相关的系统行为必须在此预告：安装文档明确提醒，通过 Docker 发布端口时，这些端口会绕过 ufw 与 firewalld 的规则，即在宿主机防火墙看似为关闭的情况下端口实际可达。相关规则进阶（DOCKER-USER 链）见第 11.5 节。

4.2 通过环境变量注入配置

环境变量（environment variable）是容器化应用的标准配置通道：镜像只管程序与默认值，随环境而变的配置在运行时注入。这符合应用配置外部化的一般实践（应用构建一次，在开发、预发、生产注入不同配置）。

```
$ docker run -d --name app1 -e APP_ENV=dev -e PORT=5000 myapp:1.0
```

同一个镜像连续运行两个容器并注入不同配置即可获得两套行为：

```
$ docker run -d --name app2 -e APP_ENV=prod -e PORT=5000 myapp:1.0
```

配置项较多时用环境文件（`--env-file` 参数指向的文本文件，每行 `KEY=value`）：

```
# env/prod.env
APP_ENV=prod
PORT=5000
```

```
$ docker run -d --name app1 --env-file env/prod.env myapp:1.0
```

两个注意点。其一，环境变量会出现在 `docker inspect` 输出中，因此不应当用它传递敏感信息（密码、私钥）；敏感配置的承载方式见第 12.3 节的 secrets 说明。其二，命令行中的 `$`、空格等字符会先被 shell 解释，注入含特殊字符的值时用单引号包裹。

4.3 数据挂载初探

第 1 章已经指出容器的删除会连同未挂载数据一起消失。把数据「挂出」容器即数据持久化，完整方案在第 8 章，本节给出最低限度的两个示例，读者至少需掌握这两个用法。

示例一，命名数据卷（named volume，由 Docker 管理的持久存储单元）存放数据库数据：

```
$ docker run -d --name db -v db-data:/var/lib/postgresql/data -e POSTGRES_PASSWORD=devpass postgres:17
$ docker volume ls
DRIVER    VOLUME NAME
local     db-data
```

数据库文件落在 `db-data` 卷内。删除容器再重挂同一卷，数据原样保留——这正是 `docker rm` 不会删除数据卷的卷生存规则。

示例二，把宿主机目录挂进容器，用于开发时直接编辑代码（本例替换 nginx 的默认站点目录）：

```
$ docker run -d --name web -p 8080:80 -v $(pwd)/site:/usr/share/nginx/html:ro nginx
```

此为 bind mount（绑定挂载）：宿主机目录与容器路径直接对应，`:ro`（read only）表示容器内只读。开发场景下改动文件即生效，无需重建镜像。

两种挂载的适用启发先给结论：数据库等应长期留存的内部数据用命名卷，代码与配置等由人直接编辑的内容用 bind mount；完整对比与语法细节见第 8 章。

4.4 让容器持续运行与自动重启

3.2 节已说明：容器退出等于主进程退出。生产场景需要策略性保证，`--restart` 提供四种策略。

策略	行为
`no`	默认值，从不自动重启
`on-failure[:次数]`	仅在非零退出后重启；可限定最多重启次数
`always`	无论退出码如何都重启；daemon 启动时也会拉起
`unless-stopped`	与 always 相同，区别仅当容器被人为 stop 后，daemon 重启后不再拉起

两种常驻场景的判定：常驻服务（数据库、Web 服务）通常取 `always` 或 `unless-stopped`；一次性任务用 `--rm` 即可。两者相差在「人为停止」这一操作上，运维会话中更倾向 `unless-stopped`，可避免计划内停机被 daemon 自作主张拉回。

4.5 常用基础镜像参考

选择基础镜像是每个容器化判断的起点。下表列出最常用的十个。

镜像	说明	典型用途
`alpine`	约 5 MB 的最小 Linux 发行版，musl libc，busybox 工具	制作精简镜像的基础层
`busybox`	更精简的单二进制 Linux 工具集合	网络诊断、极小容器
`debian`	Docker 官方 Debian 镜像	通用基础层，向后兼容性好
`ubuntu`	Ubuntu LTS 基础镜像	运行 Ubuntu 生态的工具链
`nginx`	一键启动的 Web 服务器	静态站点、反向代理
`redis`	内存键值数据库	缓存、会话
`postgres`	PostgreSQL 数据库	关系型主库
`mysql`	MySQL 数据库	关系型主库
`python:3.13-slim`	官方 Python 镜像的 slim 变体	Python 应用容器化
`node`	官方 Node.js 运行时镜像	前端构建、Node 服务

两个选型习惯值得现在就养成。其一，优先使用带 `library/` 前缀省略写的 Docker 官方镜像（Official Images），如 `nginx`、`redis`，由 Docker 官方审核维护。其二，固定明确的版本标签（如 `postgres:17` 而非 `postgres`），避免「隔天镜像悄悄换了版本」，此习惯又为第 6 章的升级策略铺路。

4.6 本章小结

- `-p` 建立宿主机到容器的端口映射，安全上优先回环绑定，并注意发布端口会绕过宿主机防火墙规则。
- 环境变量是配置注入的标准通道；敏感信息不应通过环境变量传递。
- 命名卷保存数据库一类的内部数据，bind mount 用于代码与配置；两者语法相同、定位不同。
- `--restart` 提供服务保活策略；常驻服务常用 `always` 或 `unless-stopped`。
- 第一部分到此完成：读者已可独立完成安装、镜像管理、容器操作与服务接入。第二部分解决「打包自己的应用」这一更关键的问题。

第二部分 用好 Docker

第二部分解决一个更关键的问题：把自己的应用交给 Docker。内容包括描述构建过程的 Dockerfile、控制体积的多级构建、编排多服务的 Compose，以及网络、存储与排错三块支撑能力。

第 5 章 编写 Dockerfile 打包自己的应用

第 4 章之前，容器里运行的都是现成镜像。要把自己的应用变成可交付的镜像，需要一个可重放的构建描述，这就是 Dockerfile。本章以一个 Python Web 应用为例，覆盖从指令语法到发布仓库的完整链路。

5.1 构建流程与构建上下文

Dockerfile（Docker 构建文件）是纯文本的构建指令集，按行顺序执行，每条指令生成镜像的一个层（layer，镜像的只读增量文件集，深入机制见第 10 章）。构建命令为：

```
docker build [选项] 上下文路径
```

其中的「上下文路径」（build context）是值得专门解释的概念：构建的第一步是把该目录的全部内容（受 `.dockerignore` 过滤）发送给构建器，Dockerfile 中的 `COPY` 只能从这个上下文中取文件。上下文越大，构建越慢，因此上下文目录应当只包含构建所需内容。

`.dockerignore` 文件用于排除不需要进入上下文的内容，语法与 `.gitignore` 相近。一个通用起点：

```
.git
.gitignore
node_modules/
__pycache__/
*.pyc
.venv/
build/
dist/
.env
*.log
```

最后，约定在 Dockerfile 首行写一条语法声明指令（pipeline directive），使构建器使用 BuildKit 的最新稳定语法，本教程示例均含此行：

```
# syntax=docker/dockerfile:1
```

5.2 常用指令详解

下表给出构建一个应用镜像所需的全部常用指令。其中 `FROM` 必为第一条指令（`ARG` 除外），其余按惯例自上而下为：依赖安装在前，源代码拷贝在后（原因见 5.4 节）。

指令	作用	示例
`FROM`	指定基础镜像；决定文件系统的起点与后续依赖环境	`FROM python:3.13-slim`
`WORKDIR /app`	设置后续指令与容器进程的工作目录	目录不存在则自动创建
`COPY 源 目标`	从上下文复制文件到镜像	`COPY requirements.txt .`
`RUN`	构建期执行命令并写入新层	`RUN pip install -r requirements.txt`
`ENV KEY=value`	设置环境变量，构建期与运行期都存在	`ENV APP_ENV=prod`
`ARG`	仅为构建期的变量，默认不进入最终镜像	`ARG VERSION=1.0`
`EXPOSE`	声明容器监听端口；只是文档性声明，实际发布仍靠 `-p`	`EXPOSE 5000`
`USER`	设置后续指令及容器默认进程的运行用户	`USER appuser`
`VOLUME`	声明挂载点，运行时自动生成匿名数据卷（配合见第 8 章）	`VOLUME /data`
`LABEL`	写入镜像元数据（键值对）	`LABEL org.opencontainers.image.source=...`
`HEALTHCHECK`	定义健康检查命令（第 9 章详解）	见 9.4 节
`ENTRYPOINT`	定义入口程序；以 exec 形式书写建议	`ENTRYPOINT ["./server"]`
`CMD`	提供默认命令或 ENTRYPOINT 的默认参数	`CMD ["--port", "5000"]`

`RUN` 写长命令时用 `&&` 串联并加行尾反斜杠续行，这是层优化（第 6 章）的基础写法：

```
RUN apt-get update && apt-get install -y --no-install-recommends \
      curl ca-certificates \
    && rm -rf /var/lib/apt/lists/*
```

对初学阶段通常只用到 COPY 与 ADD 两者的取舍：`COPY` 语义单一（复制），`ADD` 额外支持自动解压本地 tar 与 URL 下载；除需要自动解压外应一律用 `COPY`，少一个特性就少一类意外。

5.3 ENTRYPOINT 与 CMD 的关系

两者的分工常被混淆，先用一张表固定这个区分。

项目	ENTRYPOINT	CMD
定位	容器的入口程序（固定不变的部分）	入口程序的默认参数（可被替换的部分）
镜像中是否必需	可选	可选；两者都缺省时容器立即退出
`docker run 镜像 参数` 中的参数	不覆盖 ENTRYPOINT	整体替换 CMD
推荐书写形式	exec 形式：`["程序", "参数"]`	exec 形式

组合语义一句话概括：最终执行的命令是 `ENTRYPOINT 内容 + CMD 内容` 的拼接。Docker 官方 nginx 镜像就是典型用法：ENTRYPOINT 固定为 nginx 启动脚本，CMD 里的默认参数可在 `docker run` 时随意替换。

验证这一语义的完整实验。写一个最小 Dockerfile：

```
FROM debian:bookworm-slim
ENTRYPOINT ["echo"]
CMD ["hello"]
```

```
$ docker build -t echo-demo .
$ docker run --rm echo-demo
hello
$ docker run --rm echo-demo world
world
```

第二个 run 的参数 `world` 替换了 CMD，ENTRYPOINT 未变。需要连 ENTRYPOINT 一起替换时，使用 3.2 节表格中的 `--entrypoint` 选项。exec 形式的每个元素必须用英文双引号包裹；写成 `ENTRYPOINT [“echo”]`（中文引号）或 `[echo]` 都会在构建或运行时报错。

5.4 构建缓存与指令顺序

构建器对每条指令做缓存（build cache）：指令与输入内容都未变化时，直接复用已有层并在日志中标 `CACHED`。缓存失效的规则只有一条却影响重大：某条指令一旦失效，其后的全部指令都要重新执行，即使后面那些指令的输入没有任何变化。

由此得到顺序设计的核心结论：变化频率越低的指令越靠前。最典型的模式是「依赖文件先拷贝、安装，源代码后拷贝」：

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
```

这样修改业务代码后，依赖安装层仍命中缓存，重新构建只在最后两步发生，节省大量时间。若把 `COPY app.py .` 放在依赖安装之前，任何代码改动都会连带重装全部依赖。

5.5 实战：打包一个 Python Web 应用

以一个最小 Flask 应用为例（Flask 为 Python 的 Web 框架）。项目含三个文件。应用代码 `app.py`：

```
import os
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from Docker"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)))
```

依赖清单 `requirements.txt`：

```
flask==3.1.*
```

Dockerfile：

```
# syntax=docker/dockerfile:1
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
USER 1000
EXPOSE 5000
CMD ["python", "app.py"]
```

逐行含义：声明语法版本；基于官方 Python 3.13 的 slim 变体（体积较小的 Debian 变体，见第 6 章）；工作目录取 `/app`；先拷贝依赖文件并安装（命中 5.4 节的缓存模式）；再拷贝源代码；`USER 1000` 让运行期不再以 root 身份运行（安全动机与细节见第 11 章）；声明服务端口；最后以 exec 形式给出启动命令。

构建：

```
$ docker build -t hello-flask:1.0 .
[+] Building 12.3s (12/12) FINISHED
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt
 => [6/6] CMD ["python" "app.py"]
 => exporting to image
 => => naming to docker.io/library/hello-flask:1.0
```

输出要点有三处：步骤逐一列出且耗时来自首次构建；重复构建时这些步骤多数标 `CACHED`；结尾给出最终镜像名。

运行并验证：

```
$ docker run -d --name hello-flask -p 8000:5000 hello-flask:1.0
$ curl http://localhost:8000
Hello from Docker
```

端口写法 `-p 8000:5000` 表示宿主机 8000 对容器内 Flask 的 5000。app.py 里绑定 `0.0.0.0` 而非 localhost 是容器网络的关键细节：容器内的 localhost 只指向容器自己（第 8.2 节），不绑定对外网络接口会导致映射后访问被拒，这是把既有应用容器化时最常见的出错点。

5.6 发布镜像到仓库

构建产物要让他人可用，需要推送到仓库。以 Docker Hub 为例，流程为登录、打标签、推送：

```
$ docker login
$ docker tag hello-flask:1.0 <用户名>/hello-flask:1.0
$ docker push <用户名>/hello-flask:1.0
```

`tag` 子命令不复制镜像内容，只是给同一内容再加一个引用名。镜像名的命名空间规则：镜像名无 `/` 时指向官方镜像（无权推送），带 `<用户名>/` 时推送到该账号下。内网环境可通过 `docker tag` 给镜像加上自建仓库地址前缀后推送（形如 `<仓库地址>/<项目>/镜像名:标签`；私有镜像服务的搭建以官方 Registry 镜像及其文档为准，本教程不展开具体步骤）。

有两个细节便于后续章节使用。其一，每次推送后 `docker image ls --digests` 能看到该镜像的 `sha256:` 摘要，摘要唯一标识内容，生产环境可锁定特定摘要拉取（第 12 章）。其二，目标仓库如 Docker Hub 支持多架构镜像（multi-arch），按 CPU 架构自动分发，对应的多平台构建方法在 13.8 节。

5.7 BuildKit 与构建命令选项

Engine 自 v23.0 起，`docker build` 默认由 BuildKit（现代构建引擎，随 docker-buildx-plugin 安装）实现，相比旧版带来依赖解析、按需拉取与更精细的缓存。本教程的构建命令全部按此默认执行，无需额外开关。

命令/选项	用途
`docker build -t 名:标签 上下文`	标准构建
`--no-cache`	禁用本次构建缓存，用于排查缓存失效导致的诡异结果
`--pull`	构建前强制重新拉取基础镜像最新内容
`--build-arg KEY=value`	给 Dockerfile 的 `ARG` 传值
`docker buildx ls`	查看可用构建器（builder，BuildKit 的执行实例）
`docker buildx build --platform linux/amd64,linux/arm64`	多平台构建（详解在 13.8 节，需要额外的构建器）

其中 `--platform` 一项在默认构建器上仅支持单平台；跨平台构建需要创建带多平台能力的构建器（`docker buildx create --use`），第 13 章给出完整流程。

5.8 本章小结

- Dockerfile 把构建过程文本化；构建的第一步是把上下文目录发给构建器，`.dockerignore` 控制上下文体积。
- 指令学习中三个重点：COPY 与 ADD 的取舍、exec 形式书写 ENTRYPOINT/CMD、两者以「入口 + 默认参数」的拼接语义协作。
- 缓存规则指令级生效；「依赖在前、代码在后」是几乎所有镜像都适用的顺序模式。
- 应用监听 0.0.0.0 才能被端口映射访问。
- 构建产物通过 `docker tag` 与 `docker push` 发布；摘要与多平台概念将在后续章节使用。

第 6 章 镜像体积优化与多级构建

5.5 节构建出的镜像通常远大于应用自身的体积。体积不仅是磁盘问题，更直接影响部署速度（拉取时间）、CI 效率与攻击面。本章按「为什么大 - 怎么小」的顺序给出系统方法，重点是多级构建与缓存挂载两种现代手段。

6.1 镜像为什么比应用更大

镜像由多个只读层堆叠而成，容器的可写层只在最顶端。层有两个特性导致「看似删掉、其实还在」：其一，各层是独立打包的，后层删除前层的文件时，前层文件仍然存在于镜像中；其二，多层镜像间共享相同的基础层，互不影响。

用 `docker history` 观察每个层的体积与来源命令（`--no-trunc` 显示未截断的完整指令）：

```
$ docker history hello-flask:1.0 --no-trunc
IMAGE          SIZE      CREATED BY
f31c2ad9...    12.8MB    COPY app.py .
3e91b0c7...    60.1MB    RUN pip install --no-cache-dir -r requirements.txt
...
```

（以上数字为示例值，随实际依赖量而不同。）排错的实用结论：发现某个 RUN 层异常大，通常是它把「先装、后删」分开写成了两个 RUN；把清理动作与安装合并进同一条命令才能让删除真正生效（6.3 节）。

6.2 选择合适的基础镜像

基础镜像决定镜像「底座」的体积与可用工具。官方镜像普遍同时提供多种变体，常见取舍如下。

变体	体积量级	特点	适用
默认（如 `python:3.13`）	数百 MB	基于 Debian/Ubuntu 全量发行，工具齐全	通用开发与实验
`-slim`	约数十至一百多 MB	精简的同一发行版，保留 glibc 与 apt	生产首选之一
`-alpine`	约几十 MB	换用 Alpine 发行版、musl libc，apk 包管理	追求最小体积且依赖兼容 musl
distroless	约数十 MB	只含应用运行所需的库，无 shell、无包管理器	配合多级构建的强安全生产镜像

alpine 变体需要谨慎：musl 与 glibc 的 ABI 不一致，部分预编译二进制（官方 PyPI 的某些 wheel、特定 Linux 发行版工具）在 alpine 上不可用或需要额外装 glibc 兼容层。依赖里包含二进制扩展时，slim 是更稳的起点。

distroless 无 shell，容器内无法 `exec` 进去调试，调试路径要提前规划（第 9 章）。本教程主线选用 slim 变体。

6.3 减少层的内容

在保持可读性的前提下，针对安装类 RUN 有两条基本策略。

第一，原地清理，而非事后补充删除：

```
# 推荐写法：安装与清理在同一层
RUN apt-get update && apt-get install -y --no-install-recommends \
      gcc curl \
    && rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*
```

```
# 常见错误：分成两层时删除基本无效
RUN apt-get update && apt-get install -y gcc curl
RUN rm -rf /var/lib/apt/lists/*
```

错误写法中，第二层的 rm 是对上一层内容的覆盖，上一层里几十 MB 的索引文件仍完整保存在最终镜像中。

第二，包管理器的最小化安装。apt 加 `--no-install-recommends`，pip 加 `--no-cache-dir`，npm 用 `npm ci --omit=dev`（安装依赖而不装开发依赖，且严格按 lock 文件）。这三项在多数项目可减去数十 MB 量级。

6.4 COPY --chmod 与 COPY --link

Dockerfile 语法 v1.2 起可用 `--chmod`，v1.4 起可用 `--link`（需 Dockerfile 首行 `# syntax=docker/dockerfile:1` 保证拉取新语法）。两者分别优化权限与缓存。

`COPY --chmod` 直接在拷贝时设置权限，替代「COPY 之后单独一行 chmod」的旧写法：

```
COPY --chmod=755 entrypoint.sh /usr/local/bin/entrypoint.sh
```

`COPY --link` 改变拷贝的实现方式：文件先拷贝进一个独立的新层，再与已有层「链接合并」。其效果是拷贝层不再依赖此前任何层的输入，即便前序指令全部修改，拷贝产物也可直接复用并叠加在新内容之上；对多级构建与基础镜像更新的场景，缓存利用率提升明显。

```
# syntax=docker/dockerfile:1
FROM python:3.13-slim AS base
COPY --link app.py /app/app.py
```

适用性提醒（来自官方文档的明确约束）：使用 `--link` 时，目标路径会整体新建为空目录再写入，不能利用目标路径上的既有符号链接语义；绝大多数场景无影响，确认无此依赖时可默认使用。若构建涉及远端缓存（`--cache-from`），`--link` 为官方推荐写法。

6.5 用缓存挂载加速构建

第 6.3 节的「装完即删」解决了镜像体积，但重复构建时仍每次重新下载所有包。`RUN --mount=type=cache` 提供第三个维度：把包管理器的缓存目录挂载为构建期专用缓存，缓存内容存在于构建器本地，不写入镜像层，也不随 `--cache-to` 导出。

通用写法（以 pip 为例）：

```
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt
```

apt 的官方推荐写法同时给出并发约束（apt 要求独占缓存目录，用 `sharing=locked` 串行化）：

```
RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    --mount=type=cache,target=/var/lib/apt,sharing=locked \
    apt-get update && apt-get install --no-install-recommends -y gcc
```

`type=cache` 的关键属性汇总：`target` 为挂载路径；`id` 区分同路径不同用途的多份缓存；`sharing` 取 `shared`（默认，可并发）、`private`（每位构建者独立）或 `locked`（互斥等待）。与 6.3 节组合的典型结论是：装包用缓存挂载（加速重复构建），清理列表仍要保留（控制镜像体积），两者互不替代。

6.6 多级构建

多级构建（multi-stage build）在一个 Dockerfile 中定义多个阶段，每个阶段以 `FROM` 开始、以命名的 `AS` 标识；最终镜像只取其中一个阶段，其余阶段（编译工具链、缓存、源码）全部丢弃。这是「编译型应用镜像」的确定性优化路径，也是 distroless 基础镜像前置构建的搭配方案。

完整示例，一个 Go 程序的构建压到数十 MB 量级：

```dockerfile
# syntax=docker/dockerfile:1
FROM golang:1.25 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/root/.cache/go-build \
    go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /bin/app ./cmd

FROM alpine:3.22
RUN apk add --no-cache ca-certificates
COPY --from=build /bin/app /app
USER 1000
ENTRYPOINT ["/app"]
```

结构要点：第一阶段 `build` 用带完整工具链的官方 Go 镜像完成编译（并演示了 6.5 节的 go build 缓存挂载）；第二阶段从第一阶段仅取走单一二进制文件；基础层换成 alpine 并补齐运行时最常见依赖 ca-certificates（HTTPS 请求所需）。多级构建的默认采纳建议：凡源码需要编译或构建工具链的应用（Go、Rust、Java、前端打包），一律采用此模式。

6.7 清理与观测

构建过程中还会产生与镜像并存的两类占用：构建缓存（BuildKit 的可复用中间产物，含 6.5 节的缓存挂载内容）与悬空镜像。可用命令如下。

命令	作用
`docker builder prune`	清理构建缓存（默认保留最近使用的一份；`-a` 全部清空，含正在使用项）
`docker image prune`	清理悬空镜像
`docker buildx du`	查看构建缓存占用明细
`docker system df -v`	按对象罗列磁盘占用明细

建议的周期性操作是 `docker system df` 观察后，按对象分别清理，而非直接执行第 3 章的 `system prune -a --volumes`。

6.8 本章小结

- 镜像大的根本原因是层不可单独修改：分层的安装与删除组合会永久保留被删内容；观测工具是 `docker history`。
- 基础镜像选型口径：通用看 slim，极致体积看 alpine（注意 musl 兼容性），安全优先看 distroless（配合多级构建）。
- 体积三件套：清理并入安装层、包管理器最小化选项、多级构建只带走目标产物。
- `COPY --chmod` 替代单独 chmod 层，`COPY --link` 改善跨构建缓存复用。
- `RUN --mount=type=cache` 让包缓存留在构建器本地，与镜像瘦身正交；其内容不随 `--cache-to` 导出。

第 7 章 用 Compose 组装多服务项目

真实应用很少只有一个容器：Web 服务、数据库、缓存、消息队列的组合才是常态。为每个容器手工执行 `docker run` 并维护它们之间的网络与启动顺序，难以持续维护。Compose 以一个 YAML 文件描述整个多容器应用，以一条命令完成起停。

7.1 Compose 是什么

Compose 是 Docker 官方的多容器编排工具，面向单机环境（多主机编排见第 12 章 Swarm）。CLI 演进直接给结论：Python 版 `docker-compose`（Compose v1）已于 2022 年停止维护，现行的是 Go 实现，以 Docker CLI 插件形式提供，命令为 `docker compose`（空格，不是连字符）；2025 年发布的 Compose v5 在功能上与 v2 等价，主要面向 SDK 集成。本教程统一使用 `docker compose`。

Compose 文件按 Compose Specification 解释，该规范合并了历史上的 file format 2.x 与 3.x。两个由此产生的写作规范：顶层 `version` 字段已废弃（obsolete），写了会打警告，直接删去；文件名优先使用 `compose.yaml`（`.yml` 与 `docker-compose.yaml` 为兼容旧名）。

Compose 运行时会给应用设立一个项目名（project name，默认为 compose.yaml 所在目录名，文件内顶层 `name` 字段或命令行 `--project-name` 可覆盖）。这一项目的所有资源（容器、网络、卷）都共享项目名前缀，是后续管理与排查的重要线索。

7.2 compose.yaml 的基本结构

最小可运行示例，两个服务各占一行定义：

```yaml
name: demo-app

services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"

  cache:
    image: redis:7-alpine
```

顶层元素按需出现：`services`（业务容器列表，必需）、`volumes`、`networks`、`configs`、`secrets`、`name`。任何服务条目里未显式声明的网络与卷，都取 Compose 自动生成的默认值：项目内默认网络互联，卷自动建在主机的 docker 管理区。

7.3 服务常用配置项

字段	作用	对应 `docker run` 参数
`image`	使用的镜像	镜像名
`build: .` / `build: {context: ., dockerfile: Dockerfile}`	构建而非拉取	`docker build`
`ports` / `expose`	发布端口 / 仅容器网络内可达	`-p` / 无
`environment`	环境变量	`-e`
`volumes`	挂载	`-v`
`depends_on`	启动顺序依赖	无（手工 sleep 的替代）
`restart: unless-stopped`	重启策略	`--restart`
`healthcheck`	健康检查	无（镜像 HEALTHCHECK 的运行期替代）
`command`	覆盖 CMD	位置参数
`user: "1000"` / `read_only: true`	指定用户 / 只读根文件系统	`--user` / `--read-only`

`environment` 支持映射与列表两种写法，映射形式可读性更好，也可用 `${VAR}` 从执行命令的 shell 或 `.env` 文件取值：

```yaml
environment:
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?POSTGRES_PASSWORD 未设置}
```

其中 `:?` 语法表示变量缺失时直接报错停止，避免带着空密码启动数据库，是生产 compose.yaml 的实用写法。

`depends_on` 的完整语义需要配合 `healthcheck` 才有意义。仅写服务名的形式只保证「启动命令已发出」；加上条件则等待对方健康状态就绪后才启动本服务：

```yaml
depends_on:
  database:
    condition: service_healthy
```

`healthcheck` 与容器 HEALTHCHECK 指令同语义，四个参数为 `test`（检测命令，数组形式）、`interval`、`timeout`、`retries`，可加 `start_period` 给应用留出启动初始化时间。五者含义：`interval` 两次检测的间隔（默认 30 s）；`timeout` 单次检测的超时（默认 30 s）；`retries` 连续失败多少次视为不健康（默认 3）；`start_period` 启动宽限期，宽限期内失败不计入 `retries`，宽限期后第一个成功即恢复健康。

7.4 常用命令

```
$ docker compose up -d          # 后台启动全部服务（自动创建网络与卷）
$ docker compose ps             # 等价于 docker ps，但列出的容器带上项目信息
$ docker compose logs -f web    # 跟踪 web 服务的日志
$ docker compose stop           # 只停止容器，保留容器、网络与卷
$ docker compose down           # 停止并删除容器与网络（默认保留卷）
$ docker compose down -v        # 连同卷一起删除
$ docker compose build          # 按 build 配置重建镜像
$ docker compose pull           # 按镜像配置拉取
$ docker compose exec web sh    # 进入某个服务的容器
$ docker compose config         # 输出合并校验后的最终配置（调试用）
```

`up -d` 在项目第二次执行时行为是增量比对：配置未变的服务不重启，变化的才重建。这是以声明式描述收敛实际状态的工作方式，也是 Compose 在日常迭代中最主要的收益来源。

`down` 不带 `-v` 时保留卷（这是默认行为，对数据安全友好但对清空环境要小心）。所有涉及数据删除的命令，执行前先 `docker compose ps` 与 `docker volume ls` 确认目标。

7.5 实战：Web 服务与数据库组合

用 Compose 重构 5.5 节的单容器，加上一个 PostgreSQL 数据服务（PostgreSQL 镜像主数据目录固定为 `/var/lib/postgresql/data`）。完整 `compose.yaml`：

```yaml
name: flask-demo

services:
  web:
    build: .
    ports:
      - "8000:5000"
    environment:
      DATABASE_URL: postgresql://app:devpass@db:5432/app
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped
    user: "1000"

  db:
    image: postgres:17
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: devpass
      POSTGRES_DB: app
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d app"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 10s

volumes:
  db-data:
```

三个值得说明的设计选择。其一，`web` 用 `build: .` 而非镜像名，每次 up 自动使用 5.5 节构建的镜像。其二，`db` 通过 `healthcheck` 定义何为就绪（`pg_isready` 为 PostgreSQL 官方的就绪检测命令），`web` 的 `depends_on: condition: service_healthy` 依赖的就绪语义是通用的（否则启动早期会因数据库未就绪而报连接失败）。其三，数据落在 `db-data` 卷，破坏性删除（`down` 不带 `-v`）不影响数据。

启动并验证：

```
$ docker compose up -d
[+] Running 3/3
 ✓ Network flask-demo_default    Created
 ✓ Volume "flask-demo_db-data"   Created
 ✓ Container flask-demo-db-1     Healthy
 ✓ Container flask-demo-web-1    Started
$ curl http://localhost:8000
Hello from Docker
```

观察输出即 7.1 节的项目前缀规律：容器名为 `flask-demo-db-1`、`flask-demo-web-1`。这里 `web` 容器访问的数据库地址 `db:5432` 不是 IP，是服务名 DNS 解析（第 8 章详解），也是 Compose 之外手工 `docker run` 之所以难以保持多容器互联的原因。

7.6 其他实用特性

四个扩展点，按使用频率排序。

多环境组合。同一 base compose.yaml 加上 override 文件，仅覆盖差异项，命令行通过 `-f` 指定叠加顺序：

```
$ docker compose -f compose.yaml -f compose.dev.yaml up -d
```

profiles。给服务加 `profiles: ["debug"]` 标记，默认启动不影响它；带 `--profile debug` 运行时才启动，适合常驻但非必需的观测组件。

Develop 与 watch（Compose v2/v5 支持）。`develop.watch` 配置可让本地文件修改同步到容器（sync）或在文件变动时重新构建（rebuild）：

```
develop:
  watch:
    - action: sync
      path: ./src
      target: /app/src
```

执行 `docker compose watch` 进入监听模式，本教程不深入展开，供有本地开发热更新需求时参考。

`.env` 文件与内置变量。compose.yaml 中可用的 `${VAR}` 会从 shell 与项目根目录的 `.env` 文件取值，敏感项避免写死在仓库内。

7.7 本章小结

- Compose 用一个 YAML 描述多容器应用；现行命令为 `docker compose`，文件遵循 Compose Specification，顶层 `version` 字段已废弃。
- 配置项与 `docker run` 参数存在一一对应表；`depends_on` 的真正就绪语义来自 `healthcheck`。
- `up -d` 与 `down` 一起构成日常的最小闭环；`down -v` 涉及数据，按确认对象的原则使用。
- 项目名前缀（容器、网络、卷同名前缀）与「服务名即 DNS 主机名」两个规律，是多容器排查的锚点。

第 8 章 网络互访与数据持久化设计

第 4 章给出了访问容器与留存数据的最低限度用法。当应用包含多个容器、需要跨主机互通或承载生产数据时，需要把网络与存储当作两个独立的设计维度处理。本章先完成网络模型，再给出存储方案的完整决策表。

8.1 Docker 网络模型概览

Linux 上 Docker 的默认网络实现是：为每个网络创建一个网桥（bridge，内核中的虚拟交换机），容器通过虚拟网卡对（veth pair）连接到网桥；容器之间的流量走网桥转发，容器到宿主机外部的流量经地址转换（NAT，网络地址转换）进行。用户平时可见的行为只有三条。

第一，`docker run` 未指定 `--network` 时，容器加入名为 `bridge` 的默认网络，拥有一个内网 IP（常见为 172.17.0.0/16 段）。 第二，`-p` 发布的端口通过 NAT 实现，因此 4.1 节的回环绑定、协议后缀等语法实质是 NAT 规则的配置。 第三，多个容器放入同一个自定义网络后，可直接用容器名互访，因为 Docker 为该网络启用了内置 DNS 解析。

8.2 五种网络模式

驱动	命令写法	特点	典型用途
bridge	默认；`--network bridge` 取默认 bridge，`--network mynet` 取自定义	端口经 NAT 发布；网络内可互访	单机容器应用的标准选择
host	`--network host`	不做隔离，容器直接使用宿主机网络栈	高性能、需要真实宿主 IP 的场景
none	`--network none`	除回环外无任何网络接口	离线批处理、安全隔离
macvlan	建网络时指定 `--driver macvlan`	容器直接获得一个与宿主机同网段的 MAC 地址	容器需被局域网直接寻址
overlay	见第 12 章 Swarm	跨多台主机的虚拟网络	集群

host 模式的两条边界必须写明。其一，Linux 之外不可用；Docker Desktop 的 host 模式是模拟的，行为有差异，跨平台项目请勿依赖此模式。其二，容器进程与宿主机共享网络栈，没有端口映射这层缓冲，安全边界等同于宿主机自身进程，公网生产环境一般不用。

8.3 自定义网络与容器间互访

自定义 bridge 网络与默认 bridge 的关键差异只有一条，但它决定了应用的正确写法：默认 bridge 内没有按容器名的 DNS 解析，容器之间只能靠 IP 访问；自定义网络内的容器可以直接用容器名与网络别名互相访问。

用实验固定这个结论。创建一个自定义网络并运行两个容器：

```
$ docker network create demo-net
$ docker run -d --name node-a --network demo-net alpine sleep 3600
$ docker run -d --name node-b --network demo-net alpine ping -c 2 node-a
PING node-a (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.1 ms
```

node-b 直接用名字 node-a 发起访问，解析结果为同网段容器 IP。对照实验：默认 bridge 中同样的名字访问会失败，因为解析不到（`ping: bad address` 一类错误），必须指定 IP。

由此得到容器互联的标准写法：为多容器组建一个自定义网络，`--network` 指走出去，`--network-alias` 给一个容器多挂几个别名（供多实例负载解析使用）；网络别名是服务发现场景的补充手段。

对照第 7 章的 compose.yaml 就能理解：Compose 为项目自动创建的自定义网络 `flask-demo_default` 之所以让 `web` 能以 `db` 访问数据库，正是自定义网络 DNS 在起作用。

8.4 网络管理命令

命令	作用
`docker network ls`	列出网络
`docker network create demo-net`	创建自定义 bridge 网络；可加 `--driver` 选其他类型
`docker network inspect demo-net`	查看网络详情（子网、网关、已连接容器）
`docker network connect demo-net web`	把已运行的容器接入网络（可逐个叠加多网）
`docker network disconnect demo-net web`	从网络断开某容器
`docker network rm demo-net`	删除网络（若有容器连接会拒绝，先移除或删除容器）
`docker network prune`	清理全部未被容器使用的自定义网络（不可恢复）

8.5 数据持久化：三种挂载方式

一切存储读写发生在容器可写层都会随容器删除而消失。Docker 为「把数据放到容器之外」提供三种机制，选型的第一原则是「明确谁需要这份数据」。

机制	英文	数据位置	典型用途	可移植性
命名数据卷	named volume	Docker 管理目录（Linux 下 `/var/lib/docker/volumes/`）	数据库、应用产生的需长期留存的数据	高，Docker 直接管理
绑定挂载	bind mount	宿主机上指定的任意路径	源码、配置文件、开发目录	中，依赖宿主机路径结构
tmpfs	tmpfs	内存	敏感或不希望落盘的临时数据	无，容器停止即消失

两个挂载语法形式都支持卷与绑定挂载，`--mount` 为更明确的形式，`-v (-volume)` 为更常见的形式。对常用场景 `-v` 已足够，需要额外选项（如以指定权限挂载）时用 `--mount`：

```
$ docker run -d --name db -v db-data:/var/lib/postgresql/data postgres:17
$ docker run -d --name app --mount type=bind,source=/opt/config,target=/etc/app,readonly myapp:1.0
```

三个挂载点行为差异的预防性说明。

其一，挂载位置的目录为空时的行为差异。挂载空卷到已含内容的目标路径：Docker 会把镜像内目标路径的默认内容「拷入」卷内作为初始数据（首挂载时一次性发生）。挂载空的宿主机目录（绑定挂载）到已含内容的目标路径：容器内看到的是空目录，镜像内的默认内容被遮蔽（隐藏）。这两个行为直接决定「换一台机器绑定挂载后应用找不到默认配置」这类现象。

其二，`ro` 与 `rw`。挂载默认读写；`ro`（read only）把容器内视角限制为只读，是配置挂载的推荐默认，数据库类挂载保持读写。

其三，SELinux 环境的 `:z`、`:Z` 后缀。RHEL、Fedora 等 SELinux 启用的系统上，容器进程受 SELinux 策略限制，读取宿主机目录需要打标签；绑定挂载时加 `:z` 表示共享标签（多容器可读），`:Z` 表示独占标签。未用时典型报错为 permission denied 类；这是 Linux 特有写法，macOS 与 Windows 的 Docker Desktop 不需要（其文件系统已通过虚拟层处理）。

8.6 数据卷管理命令

命令	作用
`docker volume create db-data`	创建命名卷
`docker volume ls`	列出卷；`-q` 只取卷名
`docker volume inspect db-data`	查看卷详情（挂载点路径、驱动、选项）
`docker volume rm db-data`	删除卷（有容器引用时会拒绝；数据不可恢复）
`docker volume prune`	清理全部未被任何容器使用的卷

`prune` 是本节最需要注意的命令。两点：其一，删除机制为「未被引用」，而停止中的容器仍算持有引用，这既是保护也可能给出错误的安全感；其二，`--filter label=xxx` 可按标签过滤，生产环境为数据卷打标签后，只清理无标签的临时卷比全量清空安全得多。与第 3 章 `docker rm` 的关系需要明确：删除容器不会自动删除它挂载过的卷；`docker rm -v` 的 `-v` 才会在删除匿名卷时一并清理，命名卷总是保留，需要显式 `docker volume rm`。

8.7 bind mount 的使用要点

绑定挂载是开发场景的主力，四个高频使用点如下。

路径绝对性。`-v` 左侧是宿主机路径，必须绝对路径（相对路径会被解释为卷名）。用命令行变量拼接当前目录是惯用法：`-v $(pwd)/src:/app/src`（Windows PowerShell 用 `${PWD}/src`）。

目录与文件都可以挂载。挂载单个文件时注意：编辑器若以「替换文件」方式保存（新文件替换 inode），容器内的旧 inode 挂载点会悬空；这解释了少数编辑场景下「容器内的文件改了没变化」的问题，通常挂整个目录而不是单个文件可避免。

compose.yaml 中的相对路径。`volumes` 里的相对路径相对 compose.yaml 所在目录解析（`./src:/app/src`），是常态化开发场景的做法，`docker run` 无此便利。

性能边界。macOS 与 Windows 的 Docker Desktop，通过虚拟机文件共享实现的绑定挂载性能低于 Linux 本机；大文件密集读写（如数据库数据目录）在这两个平台更适合用卷而不是绑定挂载，这也是 4.3 节示例对数据库用卷的直接原因。

8.8 数据备份与迁移

数据卷的备份用通用容器加 tar 的组合完成，无需安装数据库客户端。备份 `db-data` 卷到宿主机当前目录：

```
$ docker run --rm -v db-data:/from:ro -v $(pwd):/backup alpine tar czf /backup/db-data.tar.gz -C /from .
```

解释：启动 alpine 容器，只读方式挂载要备份的卷，挂载宿主机当前目录为备份写入位置，容器内把 `/from` 打包到 `/backup`。`--rm` 意味着容器一次就消失。恢复是同一思路的反向：

```
$ docker run --rm -v db-data-restored:/to -v $(pwd):/backup:ro alpine tar xzf /backup/db-data.tar.gz -C /to
```

数据库级的一致性备份（在线热备）应优先使用数据库自带的备份机制（于容器内执行管理命令导出）；文件级 tar 备份适用于先经数据库管理命令停写、再对静默卷操作的场合。原则是停写之后备份优先。

跨主机迁移即镜像与数据的成对归档：镜像用 3.1 节的 `docker image save` / `load`，卷数据用本节的 tar 归档，目标主机依次导入与恢复。

8.9 本章小结

- Docker 网络一图流：网桥 + veth + NAT；默认网络不平权，自定义网络才有按容器名的 DNS 解析。
- 五种网络驱动对应不同边界：bridge 默认、host 慎用（且 Linux 限定）、none 隔离、macvlan 直连物理网、overlay 用于集群。
- 存储选型按「谁在读写这份数据」决策：应用长期数据用卷，宿主机关联内容用绑定挂载，不落盘的敏感临时数据用 tmpfs。
- 挂载空目录的两种行为差异（卷会拷入默认内容，绑定挂载会遮蔽）是新手最容易踩现象之一。
- 数据卷的备份与迁移由通用容器加 tar 完成；对活动的数据库优先使用数据库自带的备份通道。

第 9 章 日志、监控与日常排错

容器运行起来之后，真正的难点从「怎么启动」变为「怎么判断健康与怎么定位异常」。本章给出三个层次的能力：日志的收集与限制、资源与状态的实时观测、以退出码与健康检查为核心的系统排错路径。

9.1 容器日志体系

日志收集的方式按 driver（驱动）抽象。daemon 默认使用 json-file，把容器 stdout/stderr 每行一个 JSON 对象写入本地文件，`docker logs` 只对该 driver 下启动的容器有效。常用 driver 如下。

driver	说明	适用
json-file	默认；1 行 1 对象 JSON	单机、起步期
local	自带压缩、按文件轮转的本地驱动	单机、磁盘占用敏感
journald	转发给 systemd journal	Linux 上统一走 journal 管理
syslog / fluentd / gelf	转发到外部系统	有集中式日志平台
awslogs / splunk	对应云厂商	云环境

driver 在两个层面设置：daemon.json 设全局默认（`log-driver` 与 `log-opts`），单个容器用 `--log-driver` 与 `--log-opt` 覆盖；Compose 对应在服务级 `logging` 字段。第 2 章已给过的日志轮转配置在此补充 per-container 的等价写法：

```yaml
services:
  web:
    image: hello-flask:1.0
    logging:
      driver: local
      options:
        max-size: 10m
        max-file: "3"
```

日常命令在 3.5 节已给出：`docker logs -f --tail 100 -t <容器>`；多容器项目用 `docker compose logs -f <服务>`，加时间戳与跟踪的组合是排查线上问题的起点动作。

需要知道的一个格局性事实：docker logs 只能看到「主进程与其子进程写在 stdout/stderr」的内容，容器内的第三方日志文件（`/var/log` 目录的价值）如果要进 docker logs，需要由应用或 entrypoint 转发到 stdout。

9.2 资源与状态监控

四支日常 “雷达”。与 3.5 节的基础命令相比，本节强调组合与还原现场的能力。

命令	观察维度	关键补充用法
`docker stats --no-stream`	各容器 CPU、内存、网络、块设备 IO 单帧快照	脚本化的健康巡检惯用
`docker top <容器>`	容器内进程表（含宿主机视角的 PID）	判断进程数量爆炸、僵尸进程
`docker events --filter type=container`	实时事件流：启动、停止、异常退出	排查周期性重启问题时定位重启原因
`docker system df -v`	按镜像/容器/卷/构建缓存输出的磁盘明细	磁盘告警的第一反应

排查高内存时可使用（第 12.2 节的内存上限）`docker inspect -f '{{.HostConfig.Memory}}' <容器>` 查看上限，`docker stats` 观察当前用量，判定是否长期顶格。

9.3 健康检查

容器内的主进程活着不等于服务可用（主进程陷入死循环仍是 running）。健康检查（healthcheck）给容器提供稳定的就绪语义，发现机制是周期性执行一条检测命令，返回 0 视为健康（healthy），连续多次非 0 视为不健康（unhealthy），初始化期间为 starting。

三种设置入口。Dockerfile 内（第 5.2 节）；`docker run --health-cmd`；Compose 的 `healthcheck` 字段（7.3 节完整说明）。观察三处结果：`docker ps` 的 STATUS 列（`Up 2 minutes (healthy)`），`docker inspect -f '{{.State.Health.Status}}' <容器>`（直接获得 healthy / unhealthy / starting 三态），以及事件流中的 `health_status` 事件。

`--health-cmd` 版示例：

```
$ docker run -d --name web --health-cmd "curl -sf http://localhost:5000/health || exit 1" hello-flask:1.0
```

注意 `health-cmd` 只是 daemon 侧驱动，应用本身需要提供 `/health` 这类检测端点；两者一起设计才构成完整健康机制。未定义 healthcheck 的容器该值为 none，`docker inspect` 可确认。

9.4 排错的标准路径

异常大多落入有限的几类，按固定次序排查可以避免随意跳步骤。以「容器退出 / 重启 / 异常」为最常见出发点的标准路径如下。

第一步，定性。`docker ps -a` 记录状态与退出码；`docker inspect -f '{{.State.Status}} {{.State.ExitCode}} {{.State.Error}}' <容器>`。退出码到含义的映射见 3.3 节表格；此处补充两个高频解读：137 常对应内存上限触发 OOM，143 是 stop 所致的正常终止。

第二步，看输出。`docker logs --tail 200 <容器>` 取崩溃前的最后片段；大多数应用错误到这里已经水落石出。

第三步，重现进程语义。对交互式问题，`docker run` 进入容器重放启动命令（用 `--entrypoint` 换 shell，如 `--entrypoint bash`），观察启动路径上的文件、环境变量是否与预期一致。

第四步，查 daemon 日志。daemon 层报错（挂载失败、网络接口创建失败、权限等）不进容器日志，需在宿主机 journal 收集：`journalctl -u docker.service --since "10 min ago"`。

第五步，对照配置。`docker inspect <容器>` 逐段核对挂载、端口、env、restart 策略，与预期 diff。Compose 场景用 `docker compose config` 查看合并后的声明值。

9.5 常见错误与处理

将高频报错汇总为一张表，作为团队内部的速查清单。

现象	可能原因	处理
container name "/web" is already in use	同名容器已存在	`docker rm web` 后重建；或换名
port is already allocated	宿主端口被占用或已有容器映射同端口	换宿主端口；`ss -ltnp` 找占用者
cannot connect to the Docker daemon	daemon 未启动或 socket 权限	`systemctl start docker`；确认 socket 权限
permission denied（绑定挂载）	SELinux 标签不足；宿主机文件权限不匹配	绑定挂载加 `:z`（SELinux 环境）；核对 uid 映射
no space left on device	磁盘耗尽（镜像、缓存、卷、日志）	`docker system df -v` 定位后按 6.7 节清理
exec: "bash": executable file not found	使用的镜像没有 bash	基础镜像用 `sh`（如 alpine）
OCI runtime create failed	run 参数与运行时不匹配（cap/fusermount 等）	核对 `docker run` 参数，去掉不必要的特权项
image not found	拉取地址错误（命名空间/标签）	核对镜像名、登录私有仓库的推送权限

9.6 本章小结

- 日志能力的边界来自 driver 默认值；`docker logs` 只覆盖 stdout/stderr，应用写本体日志文件的需显式转发。
- 轮转不是默认行为，尽早设置；第 2 章的 daemon.json 与本章 Compose 服务级 logging 两个层面都应调整。
- 观测按维度组合：stats 看资源、top 看进程、events 看事件、system df 看磁盘。
- healthcheck 的意义是给「进程活着」与「服务可用」之间补上判定。
- 排错五步法：定性（状态与退出码）→ 看日志 → 进容器重放 → journal 查 daemon → inspect 对照声明配置。

第三部分 用精 Docker

第三部分回答两个问题：容器背后的机制是什么，如何把 Docker 用到生产。原理部分不求内核级细节，目标是让排错与配置决策有依据；生产部分给出从单机到集群的部署要点，并以完整项目收束。

第 10 章 容器运行时原理

`docker run` 的一瞬间到底发生了什么。理解原理并不需要读内核代码，收益是具体的：排查卡在某一步的容器时，能判断问题出在哪一层；配置资源与安全选项时，知道每一项对应内核里的哪个机制。

10.1 总体架构与调用链

Docker 采用客户端-服务端分层架构，组件间以 REST 接口通信。

组件	进程形态	职责
Docker CLI	`docker` 命令进程	解析命令，向 daemon 发请求
Docker daemon	`dockerd` 长驻进程	管理 API、对象（镜像/容器/卷/网络）、调度各组件
containerd	长驻守护进程	负责镜像内容存储与容器生命周期管理
容器运行时	按需进程（如 runc）	按容器配置实际创建被隔离的进程

调用链自 CLI 起：`docker run` 解析为 REST 请求 → daemon 校验并组装容器配置 → 交给 containerd 创建容器 → containerd 调用运行时（runc）建立隔离环境 → 运行时启动主进程并退出，容器由 containerd 事后接手监控。抽象出三条朴素结论。

第一，「容器即进程」。每个容器在宿主机上就是一个或一组普通进程（`docker top` 能直接看到），隔离只是给这组进程戴上的「滤镜」。 第二，镜像管理已收敛到 containerd 的镜像存储。Engine v29 起，containerd 镜像存储成为新安装环境的默认（此前长期由 daemon 内部实现管理镜像），既有数据在升级时提供迁移路径。 第三，运行时退出后容器继续运行：runc 属于「一次性调用」组件，它的退出只代表隔离环境已就绪。

两个版本相关的补充。其一，Engine v29.7.0 引入了实验性的 `embedded-containerd` 特性，把 containerd 内嵌进 daemon 进程以减少进程间开销，属实验特性，生产启用需谨慎。其二，通过 `docker info` 观察 `Storage Driver` 相关字段的输出，可确认当前环境实际使用的镜像存

储形态。

10.2 Namespace：隔离的实现

Namespace 是 Linux 内核提供的资源视图隔离机制，每个 namespace 内的进程对某类资源有独立的视图。容器由多个 namespace 共同组成，主要类型如下。

Namespace	英文	隔离内容	对容器行为的体现
MNT	mount	文件系统挂载点	容器看到的根文件系统是独立的一份
PID	process	进程编号	容器内看不到宿主机其他进程，容器内 PID 1 是自己的主进程
NET	network	网络栈、端口、路由	独立网卡与端口空间，依赖 8 章的网桥互通
UTS	UNIX timesharing	主机名	`--hostname` 设置的主机名
IPC	inter-process	System V IPC、共享内存	与宿主机不互通的进程间通信
USER	user	用户与组标识	容器内 root 与宿主机用户的映射基础
CGROUP	control group	控制组本身的视图	与 10.3 节的机制独立

USER namespace 需要额外说明：Docker 默认不开启 USER namespace 重映射（userns-remap），即容器内 root 默认就是宿主机 root 的内核态身份。默认行为加上「以 root 运行应用」是滥用 root 权限的基础，第 11 章给出处理路径。

10.3 cgroup：资源的实现

Namespace 只隔离「看见什么」，不限制「用多少」。让一组进程的 CPU、内存、IO、进程数受限，靠的是 Linux 控制组（cgroup）机制。

cgroup v2 是当前主流的统一层级版本。Docker 的资源参数自动映射到相应控制器；此处只需理解映射关系，即可读出 3.3 节参数表背后的机制。

参数	内核机制
`--cpus 1.5`	cgroup CPU 控制器的配额，跨若干核「按比例分配」
`--memory 512m`	cgroup 内存控制器的硬限额；超出触发 OOM
`--pids-limit 100`	cgroup 进程数限额；用于防 fork 炸弹
`--device-read-bps`	块设备 IO 带宽限制

版本行为：cgroup v1 的支持自 Engine v29.0 起标记为废弃，官方计划在 2029 年 5 月之前保持支持，后续主版本可能移除；使用较老的宿主机升级 Engine 前用 `docker info` 的 `Cgroup Version` 字段确认环境版本。

10.4 联合文件系统与镜像分层

镜像由多个只读层叠加而成，容器在此基础上获得一个联合文件系统（union filesystem）。主流实现是 OverlayFS：多个只读目录（lowerdir，即镜像层）叠加一个可写目录（upperdir，即容器可写层），挂载后的合并视图（merged）就是容器看到的根文件系统。工作目录（workdir）为 OverlayFS 的内部临时目录。

两个核心行为的原理解释。

写时复制（copy-up）：修改 lowerdir 中的文件时，先把它完整复制到 upperdir 再改。初次写入大文件（如多 GB 的数据文件）会显著变慢，这是把数据写入可写层的运行时性能陷阱，也是数据库数据一律走卷（第 8 章）的原理级理由。

层删除的不可见性：upperdir 中删除 lowerdir 的文件只是「盖住」，下层内容依然占空间；这解释了 6.1 节的分层安装行为。

镜像存储的两种形态在这里汇合。daemon 传统存储使用 `overlay2` 存储驱动；containerd 镜像存储下则以快照器（snapshotter）方式工作，默认同样是 OverlayFS 后端。两者对上层 Docker 命令行为基本透明，仅对底层机制的细节（如 `docker diff` 的行为细节）有所差异，遇到特殊场景以 `docker info` 的输出为准。

10.5 镜像的内部结构

一个镜像从仓库侧看，有四类内容对象。

内容对象	说明
索引（index/manifest list）	按架构列出可用镜像清单（多平台镜像的关键）
manifest	描述一个镜像由哪些层组成、层关系如何
配置（config）	镜像的环境变量、启动命令、层记录等元数据
层（layer blobs）	每层二进制的实际内容，按 digest 寻址

所有内容按摘要（digest）寻址，任何一层修改都会触发配置与 manifest 的重算，这与 6.1 节的缓存与共享逻辑同源。使用时即可从 `docker image ls --digests` 看到 digest：该值随内容唯一，与所在仓库的标签无关，也是第 12 章锁定摘要的依据。历史上使用的镜像清单 schema v1 已于 Engine v28.2 起不再支持，旧标签的私有镜像升级前需重新构建。

10.6 容器进程与生命周期底层

容器主进程在容器内是 PID 1，承担两个宿主机上不存在的职责：其一，作为进程组负责人，收到 kill 等信号时负责转发给子进程；其二，作为孤儿进程的收养者。很多应用没有按 PID 1 的语义编写，表现为 stop 后进程迟迟不退（终被升级为 SIGKILL）或僵尸进程累积。两条实务结论。

其一，主机内 `--memory` / `--pids-limit` 等参数之外，`docker run --init` 会为容器附带一个轻量初始化进程（内部使用 tini），专门处理信号转发与收养；对启用大量进程、信号行为不标准的应用是低成本的修复手段。 其二，调试判断僵尸进程时，回到 9.2 节 `docker top`，pid 观察加 `-Z` 附加标记在宿主机视角会缺失，以 `docker top` 直接观测为准。

10.7 Engine API 与版本协商

CLI 与 daemon 之间的一切交互都走版本化的 REST 接口（Engine API）。客户端会自动协商可用版本：daemon 支持一系列历史 API 版本以保证旧客户端可用；Engine v29 把最低支持的 API 版本提升至 1.44，过低版本的客户端将收到明确报错。

手工调用 API 在自动化集成（CI/CD、监控脚本）中有用。示例（Linux）：

```
$ curl --unix-socket /var/run/docker.sock http://localhost/v1.44/version
{"Version":"29.9.2","Os":"linux","Arch":"amd64",...}
```

通过 Unix 套接字而非 TCP 访问是只有 daemon 暴露在 socket 上的标准做法。对外暴露 TCP 需要完整 TLS，这一安全要求在第 11.5 节给出版本依据。API 的完整定义见官方「Engine API 参考」。

10.8 本章小结

- 架构上「容器即进程」：daemon、containerd 与运行时各司其职，runc 是按需调用的一次性组件。
- Namespace 解释「看到的不同」，cgroup 解释「能用多少」；默认 root 身份的问题在下一章处理。
- OverlayFS 的 copy-up 与层删除行为，是卷挂载与镜像优化两条实务线的原理依据。
- 镜像是按 digest 寻址的内容集合；schema v1 已弃用，旧镜像升级前需重新构建。
- Engine API 是 CLI 的底层通道；v29 将最低版本提升至 1.44。

第 11 章 安全与信任

容器在默认配置下并非安全边界：daemon 以 root 运行、容器内进程默认拥有 root 内核身份、镜像来源未经校验即运行。本章按「运行时 - 供应链 - daemon」三层给出加固手段，并把安全配置落成一张检查清单。

11.1 威胁面模型

先明确风险来自哪里，再加护。

层面	风险	关键事实
运行时	容器内应用被攻破后波及宿主机	容器与宿主机共享内核；无强隔离的代码不应执行
权限	容器内以 root 运行	默认容器内 root 直接映射宿主机 root（10.2 节）
访问面	管理接口权限不当	daemon socket 获得 root 等价权限（2.3 节）
供应链	运行来路不明的镜像内容	镜像拉取默认不验证内容，来源信任依赖仓库与镜像策略
配置面	特权或敏感挂载	`--privileged` 与挂载 docker.sock 属最高风险操作

本节的目标不是勾勒防御理想态，而是给出每层的合理与不合理配置的最小判断。

11.2 运行时权限控制

从最小权限原则出发，运行时侧有四个抓手。

第一，不用 root 运行应用。两个实施点：Dockerfile 中的 `USER`（5.5 节已采用），或 `docker run --user 1000`。注意两个受影响的常见点：挂载目录的属主与该用户不匹配时出现 permission denied，此时优先核对挂载目录权限；使用 1024 以下端口同样受限制，映射关系改由 `-p` 在宿主机侧完成。

第二，能力（capabilities）裁剪。Linux 内核把 root 的权限拆分为一组 capabilities，容器默认获得其中一部分。常规 Web 服务通常不需要额外能力，可收紧：

```
$ docker run --cap-drop=ALL myapp:1.0
$ docker run --cap-add=NET_BIND_SERVICE myweb:1.0
```

`--privileged` 的使用需给出明确理由：它让容器接近宿主机权限，等于绕开上述全部隔离，常见正当场景仅限少数工具类需求（如自建 CI 内嵌 Docker）；其余场景禁用，改由具体的能力或设备粒度放行。

第三，只读根文件系统与增量加固选项。应用可写目录挂在卷或 tmpfs，其余只读：

```
$ docker run --read-only --tmpfs /tmp myapp:1.0
```

同时开启 `--security-opt no-new-privileges`（父进程无法通过 setuid 等机制提权），Compose 中对应 `security_opt` 字段。细节上，默认 seccomp 与 AppArmor 配置已提供一套基础降低面，改动前没有理由先关掉它们。

11.3 rootless 模式

rootless 模式（daemon 无 root 运行模式）把 daemon 本身放到非 root 用户下运行，容器与 daemon 均以普通用户权限工作，是 daemon 层级的隔离增强。适用与限制两边都要看：Docker Desktop 下 rootless 相关配置已内置，无需手工操作；Linux 上由官方 Rootless 向导提供脚本（`dockerd-rootless-setuptool.sh install`）完成配置，部分网络与挂载特性受限，具体以官方 Rootless 教程为准。

一个务实判断：多租户环境或对 root 暴露面敏感的服务器值得启用；个人开发机在 docker 组（2.3 节）+ 非 root 应用进程（11.2 节）的组合下，rootless 的边际收益有限，可优先解决应用侧的非 root 运行。

11.4 镜像供应链安全

供应链层面有三件事。

其一，只运行来源可信的镜像。优先 Docker 官方镜像（如 4.5 节所述），或 Docker Hub 上带 Verified Publisher 标记的发布者；Docker Scout 提供镜像自身的发现（漏洞、包构成）与策略评估能力（官方产品文档），CI 中可把扫描设为阻断门禁。

其二，锁定内容而非标签。生产依赖镜像按下摘要拉取，即取 manifest 内容指纹为准（第 10.5 节）：

```
$ docker pull redis@sha256:0b1c9f2a4d...   # 摘要以 docker image ls --digests 查得
```

其三，建立对 Docker Content Trust（DCT）现状的明确认知，避免按旧文档操作。DCT 与其 Notary v1 服务（notary.docker.io）已被 Docker 官方完全退役（官方于 2026 年 6 月发布退役完成说明，核实于本教程的核实日期）；基于 DCT 签名与验签的构建、分发流程需要迁移。目前官方主推 Docker Scout 的策略评估能力，生态中亦有 sigstore 与 cosign 等第三方镜像签名方案可选；本教程记录该事实与替代方向，替代方案的具体实现以各自官方文档为准。

11.5 Docker daemon 与 API 安全

daemon 层有三个必须刻意的点。

暴露面控制。默认只监听本机 Unix 套接字，这是多年的默认形态；远程管理优先使用 SSH 通道或 Docker Desktop 提供的远程能力。若选择 TCP 暴露（配置 `tcp://0.0.0.0:2375` 等），则必须配 TLS：自 Engine v28.0 起，未认证的 TCP 连接（未配 TLS）已被官方移除，此前的默认放行行为不再存在。

防火墙联动。8 章已预告：Docker 发布的端口会旁路 ufw 与 firewalld 规则。正确姿势是把需要控制暴露面的规则加入 `DOCKER-USER` 链（daemon 文档说明的自定义拦截点），而不是「以为宿主机防火墙挡住了就没问题」。

镜像加速与内网镜像仓库。daemon.json 的 `insecure-registries` 可让指定地址走不加密的仓库通信；按内网环境配置为限定的仓库地址，不要泛化为 `*` 或写通配域名。

11.6 安全检查清单

按启用难度递增的顺序，供阶段对照自评。

项	说明	依据
应用不以 root 运行	Dockerfile `USER` 或 `--user`	11.2 节
只挂载必要的目录与 sock	未使用的 docker.sock 挂载一律移除	11.1 节
发布端口最小化，优先 127.0.0.1 绑定	公网服务经反向代理	4.1、11.5 节
生产镜像固定版本或摘要	不用 `latest`	10.5、11.4 节
容器加固参数基础组合	`--cap-drop/ADD`、`--read-only`、`no-new-privileges`	11.2 节
日志轮转与磁盘限额	9.1 节 daemon.json	防止日志拖垮磁盘
daemon 不暴露未认证 TCP	远程访问优先 SSH	11.5 节
镜像扫描纳入流程	Docker Scout 或等价能力做门禁	11.4 节
rootless / 更强隔离按场景评估	多租户或严格环境启用	11.3 节

11.7 本章小结

- 容器不是沙盒：内核共享决定了它适合承信代码，不适合跑不受信程序。
- 运行时侧按掉 root、裁分支能力、只读与 no-new-privileges 四点收紧；`--privileged` 默认禁用。
- rootless 是 daemon 层的层级内存提升，Linux 上按官方向导启用，按环境价值取舍。
- 供应链侧锁定摘要、开启扫描门禁；DCT/Notary 已退役，旧文档流程不再有效。
- daemon 层面：socket 与 TCP 暴露是两个最高权重决策，防火墙联动要用 DOCKER-USER 链落实。

第 12 章 生产部署

从「能跑」到「可靠的对外服务」，需要补上三类工程决策：资源的硬性约束、数据与配置的安全通道、多主机与升级的组织方式。本章以单机生产为基线，以 Swarm 为多主机的最小方案，最后给出生产检查清单。

12.1 生产环境的目标

给生产环境列出可验收的目标，而不是直接堆配置：服务能按预期重启并自动恢复；资源异常时受控（不冲垮宿主机）；数据在容器重建后仍在，且有可执行的备份与恢复路径；配置变更可追溯（全部落在受版本管理的文本中）；升级可滚动、可回退。本章各节逐项对应这些目标。

12.2 资源管控与进程治理

单机上最重要的两项治理是内存与进程数上限，两者直接决定「一个失控容器不能拖垮宿主机」。

参数	作用	典型取值逻辑
`--memory 1g`	内存硬上限（cgroup 限额）；超限触发 OOM 结束进程（对应退出码 137）	应用峰值内存乘 1.5 至 2 的余量
`--memory 1g --memory-swap 1g`	两者相等即完全关闭 swap 使用	严格环境使用，避免换页隐藏内存问题
`--cpus 1.5`	CPU 配额上限	按可容忍的 CPU 占用设定
`--pids-limit 200`	容器内进程数上限	防 fork 炸弹的默认保险
`--ulimit nofile=65536:65536`	文件句柄上限运行时设置	高并发服务常见需求

配合驻留服务运行的策略是 `--restart unless-stopped`（4.4 节分析两者差异）；`on-failure[:N]` 适合加限量重试的后台任务。daemon 层面还有一项与生产体验直接相关的配置：`live-restore`。在 daemon.json 中开启 `"live-restore": true` 后，升级或重启 daemon 不再停止运行中的容器，是长驻服务升级 daemon 窗口的标准手段。

监控侧依据 9.2 节的 `docker stats` 巡检，重点观察内存是否长期顶格（顶格即 OOM 风险态），并以此反馈调整上限。

12.3 数据与配置的安全通道

三块，从数据到配置。

数据持久化与备份。命名卷承载全部需留存的数据（8.5 节），8.8 节的 tar 组合命令作为备份；生产增值两点：为数据卷打标签（`docker volume create --label daily-backup` 之类），使清理与备份脚本可用 `--filter label=` 圈定对象；把 8.8 节的备份命令放入定时任务（cron 或 systemd timer），并按「恢复演练」的纪律定期验证归档可用。未演练过的备份不算备份。

配置与敏感信息。环境变量承载普通配置（4.2 节）。敏感信息（密码、令牌）不应出现在镜像、compose.yaml 或 shell 历史中。Docker 生态提供的通用通道是 secrets：Compose 文件顶层声明 secrets，运行时以文件形式挂载（默认路径为容器内 `/run/secrets/<名称>`），由应用读取文件而非读环境变量：

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d app"]
      interval: 5s
      timeout: 3s
      retries: 10

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

宿主机侧的 secrets 文件目录应收敛权限（仅管理账号可读），compose.yaml 与密钥文件分开存放于版本管理之外。镜像侧还有一条预防：确保构建上下文不含 `.env` 或密钥文件（第 5 章的 `.dockerignore` 已包括 `.env`）。

日志治理。按 9.1 节设置轮转上限，并指定后端（本机 local 或直连平台），避免磁盘被拖垮这一常见的次生故障。

12.4 跨主机部署：Swarm 简介

多主机场景从简单的需求出发：服务实例分布在多台机器上，单机故障时不中断服务。Docker 自带的 Swarm 模式正是这一层能力：把多台装好 Docker 的机器组成集群，以 service（服务）为单位声明期望状态，daemon 自动在节点间调度与维持副本数。

最小上手流程（以三节点为例，命令在管理节点与工作节点分别执行）：

```
$ docker swarm init --advertise-addr <管理节点IP>
$ docker node ls
$ docker service create --name web --replicas 2 --publish published=8080,target=80 nginx
$ docker service ps web
```

`swarm init` 输出 join token，集群组建后 `docker service create` 即可在集群内任意调度。它的几个天然特性：内部网络自动使用跨主机互通的 overlay 网络驱动；`--publish` 走路由网格（routing mesh，任何节点到达该端口都会被路由到该服务实例），随时随地可访问；管理节点间证书轮转自动完成。

扩缩容、滚动更新、回退都是 service 上的操作：

```
$ docker service scale web=4
$ docker service update --image nginx:1.27-alpine web
$ docker service rollback web
```

Swarm 还有更多细节（以官方文档为准）：管理与工作节点的角色划分、`docker node promote/demote` 的用法、更细粒度的调度约束与网络加密选项。

Compose 文件在 Swarm 中同样可用（`docker stack deploy -c compose.yaml 应用名`），但 Compose Specification 中一部分开发向字段（如 `develop.watch`）在 Swarm 端不生效，部署前应核对字段支持度。

12.5 滚动更新策略

以镜像升级这一最高频操作为例，Swarm 侧的控制参数是 `--update-*` 系列：

```
$ docker service update --image myapp:2.0 \
    --update-parallelism 1 --update-delay 10s \
    --update-failure-action rollback web
```

含义逐条说出：一次只更新一个副本（parallelism 1），每批之间隔 10 秒（delay 10s），更新失败时自动回退（failure-action rollback）。与 7 章的 Compose `up -d` 对照：单机场景 Compose 重建即「全量切换」，Swarm 的价值就在这批分组与失败处理策略上。

12.6 何时选择 Kubernetes

随规模增长会出现「不止三台主机、多套应用、跨团队使用」的阶段，Swarm 的内置调度与网络编排到此够用的判断开始转变：需要更复杂的负载均衡、多租户隔离、更丰富的社区生态（Prometheus、Ingress、Helm 等体系）时，Kubernetes 是当下事实上的平台标准。务实的判断区间：容器化改造与单机生产使用 Compose 即可；3 至 5 台主机的小规模服务可用 Swarm 起步；规模增长后再评估迁移 Kubernetes。迁移的全部成本集中在发布流程与配置格式，容器与镜像本身完全复用，这也是全书传导的方法论：把应用收敛为开放的镜像资产，编排层随规模升级而更换。

12.7 生产部署检查清单

项	依据
镜像固定版本或摘要，禁用 `latest`	第 5、10 章
全量资源上限：memory / cpus / pids-limit	12.2 节
数据卷化并打标签，备份任务定时执行	12.3 节
敏感配置走 secrets 文件，不进环境变量与镜像	12.3 节
日志轮转与上限配置齐备	9.1 节
restart 策略明确，与目标行为匹配	4.4 节
暴露端口走回环绑定或反向代理，防火墙联动用 DOCKER-USER	4.1、11.5 节
镜像扫描在 CI 中作为门禁	11.4 节
多主机需求出现时才引入 Swarm / Kubernetes	12.4 节
生产环境对接监控告警（至少依赖 stats/events 的外部采集）	9.2 节

12.8 本章小结

- 生产四要素：资源硬上限、数据可恢复、配置可追溯、升级可回退。
- 资源管控由 cgroup 的三个上限参数承载；`live-restore` 让 daemon 升级不再中断容器。
- 敏感信息走 secrets 文件体系；构建上下文与版本管理中排除所有密钥。
- Swarm 是多主机的最小方案，路由网格与滚动更新是两个即得能力；规模超出单集群承载再评估 Kubernetes。
- 检查清单逐项对应前文，是交付前的对照表。

第 13 章 综合实战：容器化一个真实项目并发布

前十二章的技法在这一章里串成一条完整路线：为一个带数据库的 Web API 建立容器化工程（含 Dockerfile、Compose、init 脚本与 `.dockerignore`），完成本地构建与验证、数据备份演练、迭代升级，最终以多平台构建推送到 Docker Hub。

13.1 项目背景与结构

项目为一个待办事项接口服务（待办事项 API）：用户可以增查条目，数据持久于 PostgreSQL。最终目录结构全貌如下：

```
todo/
├── db-init/
│   └── 01-schema.sql        # 数据库初始化脚本
├── secrets/
│   └── db_password.txt      # 数据库口令（不入版本管理）
├── src/
│   ├── app.py               # 应用代码
│   └── requirements.txt     # Python 依赖
├── compose.yaml
├── Dockerfile
└── .dockerignore
```

按容器化改造的一般顺序推进：定结构（本节）与写应用（13.2 节）本来就已存在，容器化的增量是 Dockerfile、compose.yaml 与 .dockerignore 三个文本，本轮先建立目录。

13.2 编写应用与依赖清单

应用监听 8000（绑定 0.0.0.0，5.5 节的要求），提供两个接口：`GET /health` 健康检查（查一次库以同时验证联通），`GET/POST /tasks` 列表与新增。`src/app.py`：

```
import os
from flask import Flask, request, jsonify
import psycopg
from psycopg.rows import dict_row

app = Flask(__name__)

def get_conn():
    return psycopg.connect(os.environ["DATABASE_URL"], row_factory=dict_row)

@app.get("/health")
def health():
    try:
        with get_conn() as conn:
            conn.execute("SELECT 1")
    except Exception:
        return jsonify(status="unhealthy"), 503
    return jsonify(status="ok")

@app.route("/tasks", methods=["GET", "POST"])
def tasks():
    if request.method == "GET":
        with get_conn() as conn:
            rows = conn.execute("SELECT id, title, done FROM tasks ORDER BY id").fetchall()
        return jsonify(rows)
    data = request.get_json()
    title = (data or {}).get("title", "").strip()
    if not title:
        return jsonify(error="title is required"), 400
    with get_conn() as conn:
        row = conn.execute(
            "INSERT INTO tasks (title) VALUES (%s) RETURNING id, title, done",
            (title,),
        ).fetchone()
        conn.commit()
    return jsonify(row), 201

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

`src/requirements.txt`：

```
flask==3.1.*
psycopg[binary]==3.*
```

表结构脚本 `db-init/01-schema.sql`（PostgreSQL 官方镜像在首次初始化时执行 `/docker-entrypoint-initdb.d/` 目录下的 SQL，这是初始化建表的标准通道，按官方镜像文档核实）：

```
CREATE TABLE IF NOT EXISTS tasks (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    done BOOLEAN NOT NULL DEFAULT false
);
```

密钥文件 `secrets/db_password.txt` 采用生成强口令后写入文件，并确认其属主权限仅供管理账号读取。

13.3 编写 Dockerfile

应用第 5、6 章的全部优化结论：slim 基础镜像、依赖先装、源码后拷贝、pip 缓存挂载、非 root 运行、CMD 用 exec 形式、加 HEALTHCHECK：

```
# syntax=docker/dockerfile:1
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt ./
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt
COPY app.py ./
USER 1000
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --retries=5 --start-period=10s \
  CMD python -c "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://127.0.0.1:8000/health').status == 200
else 1)"
CMD ["python", "app.py"]
```

`.dockerignore`（覆盖 5.1 节清单并补上本章新目录）：

```
.git
.gitignore
__pycache__/
*.pyc
.venv/
.env
secrets/
db-init/
compose.yaml
Dockerfile
```

两个取舍说明：`db-init/` 与 `secrets/` 均由挂载通道进入容器（分别挂入数据库 init 目录与 secrets 文件），无需进入应用镜像；`compose.yaml` 与 `Dockerfile` 本身不属于运行期内容。排除它们同时减小上下文与泄露面。

13.4 编写 compose.yaml

承载三块内容：应用与数据库两个服务、各自的健康依赖、数据卷与 secrets。文件名按第 7 章约定取 `compose.yaml`，顶层不写 `version`：

```yaml
name: todo

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://app:${POSTGRES_PASSWORD:?POSTGRES_PASSWORD 未设置}@db:5432/app
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:17
    environment:
      POSTGRES_USER: app
      POSTGRES_DB: app
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - db-data:/var/lib/postgresql/data
      - ./db-init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d app"]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 10s
    restart: unless-stopped

volumes:
  db-data:

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

四个对照设计（来源章节已注明）：非 root 与只读挂载（11.2、8.5 节）；密码走 secrets 文件与 `POSTGRES_PASSWORD_FILE` 镜像特性（12.3 节，该环境变量为 PostgreSQL 官方镜像文档的既定支持）；数据库初始化脚本只读挂载（官方镜像的 init 目录机制）；`${POSTGRES_PASSWORD:?}` 缺失即报错（7.3 节）。

13.5 构建与验证

首次构建建议先不加 `-d` 运行 `docker compose up --build`，观察完整构建与启动日志；确认服务稳定后按同参数加 `-d` 切入后台运行：

```
$ POSTGRES_PASSWORD=devpass docker compose up -d --build
 ✓ Network todo_default        Created
 ✓ Volume "todo_db-data"       Created
 ✓ Container todo-db-1         Healthy
 ✓ Container todo-api-1        Started
```

依序验证四个点：容器状态（`docker compose ps`，STATUS 应为 healthy 与 running）；健康接口；新增与列表；数据卷的持久化：

```
$ curl http://localhost:8000/health
{"status":"ok"}

$ curl -X POST http://localhost:8000/tasks -H "Content-Type: application/json" -d '{"title":"撰写 Docker 教程"}'
{"id":1,"title":"撰写 Docker 教程","done":false}

$ curl http://localhost:8000/tasks
[{"id":1,"title":"撰写 Docker 教程","done":false}]
```

持久化验证做一次完整的破坏流程：`docker compose down` 删除容器（保留卷）后重新 `up -d`，用 curl 复核 `id=1` 的条目仍在（密码沿 13.4 节 `${POSTGRES_PASSWORD:?}` 的声明直接在命令前提供）：

```
$ docker compose down
$ POSTGRES_PASSWORD=devpass docker compose up -d
$ curl http://localhost:8000/tasks
[{"id":1,"title":"撰写 Docker 教程","done":false}]
```

这一步验证的正是 12.3 节的目标「数据在容器重建后仍在」。

13.6 日常迭代与升级

修改代码之后的两条固定命令（第 5 章已给出对应原理：改动只使后置层失效）：

```
$ docker compose build api
$ docker compose up -d
```

等价的合并写法为 `docker compose up -d --build`；Compose 会比对镜像与构建上下文的差异，只重建相关服务（api），数据库保持不动。镜像标签可由 `build` 配置或命令行 `-t` 指定（`--build-arg` 与 `--push` 的场景见 13.8 节）。

13.7 数据备份与恢复演练

按 12.3 节的要求执行一次真实演练（备份与恢复已在 8.8 节给出命令）。备份：

```
$ docker run --rm -v todo_db-data:/from:ro -v $(pwd):/backup:rw alpine tar czf /backup/todo-db-$(date +%F).tar.gz -C /from .
```

恢复演练可在 `db-data-restored` 新卷上执行：按 8.8 节的恢复命令导入归档并校验内容可读，然后临时把 compose.yaml 中的卷名替换为 `db-data-restored`，执行 `docker compose up -d` 验证服务正常、数据齐全。未经过恢复验证的备份不应视为有效备份；留下一份当天验证通过的恢复命令，是这一环节的直接产出。

13.8 多平台构建与发布

发布一次双架构镜像（x86_64 服务器与 Apple silicon 开发机都用得到）。一次性创建多平台构建器（5.7 节的概念）：

```
$ docker buildx create --name multiarch --use
$ docker buildx inspect --bootstrap
$ docker buildx build --platform linux/amd64,linux/arm64 -t <用户名>/todo-api:1.0 --push .
```

`--platform` 列出两个目标架构；`--push` 完成构建后直接推送，`docker pull <用户名>/todo-api:1.0` 在另一台机器上可直接投入使用。日常小版本的发布也采用同样的 buildx 命令；服务器侧以摘要或固定标签拉取（12.4 节的 Swarm 部署同样可指定镜像版本）。

13.9 能力盘点与后续学习路径

至此完成从「没用过容器」到「独立完成一次生产容器化交付」的闭环，读者现在具备的能力清单。

能力	出处
按平台安装并验证 Docker 环境	第 2 章
日常管理镜像与容器、接入端口与环境变量	第 3、4 章
以 Dockerfile 描述构建并控制体积	第 5、6 章
用 Compose 编排多服务、设计网络与存储方案	第 7、8 章
从日志与状态判断健康，按标准路径排错	第 9 章
解释容器原理，判断配置与场景	第 10 章
按清单完成安全加固与供应链校验	第 11 章
按生产要求落实资源、数据与升级策略	第 12 章
完成一次端到端交付并于多平台发布	第 13 章

后续深入的方向有三条，各按需求选择。

构建与 CI：把 13.8 节的 buildx 命令落入自动化平台（关注 BuildKit 与 Docker Scout 集成以形成镜像质量门禁）。 Kubernetes：当集群跨多个应用与团队时把编排从 Swarm 迁出（12.6 节的迁移成本口径仍然适用）。 生态：按项目属性使用 Testcontainers（测试环境的容器化管理，官方产品）、Docker Desktop 的桌面调试能力等。

附录

附录 A 常用命令速查表

以下为前文出现的高频命令汇总，按对象分组。删除类命令已在上文标注后果，执行前先核对目标。

镜像

任务	命令
拉取 / 列出 / 删除	`docker pull <镜像>:<标签>` / `docker image ls` / `docker image rm`
查看详情与分层	`docker image inspect` / `docker history --no-trunc <镜像>`
导出 / 导入	`docker image save -o x.tgz <镜像>` / `docker image load -i x.tgz`
打标签 / 推送	`docker tag <镜像> <目标名>` / `docker push <目标名>`
查看摘要	`docker image ls --digests`
按摘要拉取	`docker pull <镜像>@sha256:<摘要>`

容器

任务	命令
后台运行 / 交互运行	`docker run -d --name <名> <镜像>` / `docker run -it --rm <镜像> bash`
端口 / 环境 / 挂载	`-p <宿主>:<容器>` / `-e KEY=value` / `-v <卷或路径>:<容器路径>`
状态与列表	`docker ps -a` / `docker inspect -f '{{.State.ExitCode}}' <容器>`
进入 / 执行	`docker exec -it <容器> bash` / `docker exec <容器> env`
日志 / 资源	`docker logs -f --tail 100 <容器>` / `docker stats --no-stream`
停止 / 删除 / 清理	`docker stop <容器>` / `docker rm <容器>` / `docker container prune`

网络与存储

任务	命令
创建 / 查看 / 删除网络	`docker network create <名>` / `docker network inspect <名>` / `docker network rm <名>`
接入 / 断开网络	`docker network connect <网> <容器>` / `docker network disconnect <网> <容器>`
卷管理	`docker volume create/ls/inspect/rm`
卷备份 / 恢复	第 8.8 节 tar 组合命令

构建与 Compose

任务	命令
构建	`docker build -t <名>:<标签> .`；`--no-cache`、`--pull`、`--build-arg`
多平台构建	`docker buildx create --use` 后 `docker buildx build --platform ...`
起停	`docker compose up -d` / `docker compose down`（含 `-v`）
状态与日志	`docker compose ps` / `docker compose logs -f <服务>`
进入 / 重建	`docker compose exec <服务> sh` / `docker compose up -d --build`
配置检查	`docker compose config`

系统维护

任务	命令
磁盘占用	`docker system df` / `docker system df -v`
清理	`docker image prune` / `docker builder prune` / `docker system prune`
事件与服务信息	`docker events --filter type=container` / `docker info`
服务管理	`systemctl status/start/restart docker`

附录 B 术语表

术语	英文	含义
镜像	image	只读的应用打包物，由多个只读层构成
容器	container	由镜像启动的运行实例，随删除而消失
仓库	registry	集中存放与分发镜像的网络服务
标签	tag	同一镜像不同版本的别名；`latest` 为默认标签
摘要	digest	镜像内容的指纹，形如 `sha256:...`，内容一致则摘要一致
层	layer	镜像的最小可复用构成单元，只读
多级构建	multi-stage build	单文件内多个构建阶段，最终镜像只取其一
构建上下文	build context	`docker build` 发给构建器的目录内容
BuildKit	—	现代构建引擎，v23.0 起 `docker build` 的默认实现
缓存挂载	cache mount	`RUN --mount=type=cache` 提供的构建期持久缓存
数据卷	volume	Docker 管理的持久存储单元，独立于容器生命周期
绑定挂载	bind mount	把宿主机目录直接映射进容器
网桥	bridge	内核虚拟交换机，单机容器网络的基础
路由网格	routing mesh	Swarm 中把端口请求路由到服务实例的机制
联合文件系统	union filesystem	只读层叠加可写层的文件系统形态，主流实现为 OverlayFS
命名空间	namespace	内核提供的资源视图隔离机制
控制组	cgroup	内核提供的资源限制机制，docker 的 `--memory` 等参数的承载者
写时复制	copy-up	OverlayFS 首次修改下层文件时先将其复制到可写层的行为
镜像存储	containerd image store	Engine v29 起新安装默认采用的镜像内容存储形态
健康检查	healthcheck	判定容器内服务是否就绪的检测机制
重启策略	restart policy	`--restart` 指定的容器自动重启规则
Swarm 模式	swarm mode	Docker 内置的多主机编排能力
容器编排	orchestration	对多容器应用的部署、扩缩与自愈的管理
secrets	—	以文件形式注入容器的敏感配置通道

参考资料与核实声明

本教程全部版本号、命令行为与版本差异说明的核实日期为 2026-10-21，来源为 Docker 官方文档（docs.docker.com）与官方发布说明。以下文档路径在阅读本教程时可用作延伸查证。

主题	官方文档路径
Linux 安装	docs.docker.com/engine/install/
Engine 版本说明	docs.docker.com/engine/release-notes/
废弃特性汇总	docs.docker.com/engine/deprecated/
Dockerfile 参考	docs.docker.com/reference/dockerfile/
Compose 文件规范	docs.docker.com/reference/compose-file/
Swarm 模式	docs.docker.com/engine/swarm/
Rootless 模式	docs.docker.com/engine/security/rootless/
Docker Scout	docs.docker.com/scout/
Docker Desktop 安装	docs.docker.com/desktop/setup/install/

正文标注「以官方文档为准」或给出版本号行为的条目，如与最新文档不一致，以官方文档为准。